Chapter 2-

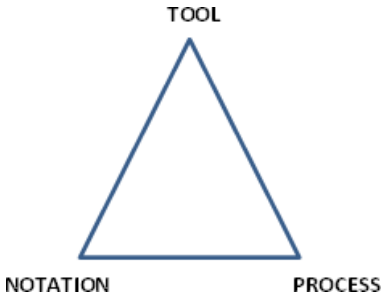
An overview of UML

Outline

- Object oriented Methodologies
- An overview of UML
- Diagrams in the UML
  - Use Case Diagrams
  - Class Diagrams
  - Sequence diagrams
  - Activity diagram
  - Deployment Diagram
  - Component diagram. . .

Object oriented Methodologies

- Many methodologies have been developed for object oriented development.
- A methodology usually includes
  - Notation : Graphical representation of classes and their relationships with interactions.
  - Process : Suggested set of steps to carry out for transforming requirements into a working system.
  - Tool : Software for drawings and documentation



An overview of UML

- The UML stands for Unified modeling language, is a standardized general-purpose visual modeling language in the field of Software Engineering.
- It is used for specifying, visualizing, constructing, and documenting the primary artifacts of the software system.
- It helps in designing and characterizing, especially those software systems that incorporate the concept of Object orientation.
- It describes the working of both the software and hardware systems.

Goals of UML

- UML came into existence after the introduction of object-oriented concepts to systemize and consolidate the object-oriented development, due to the absence of standard methods at that time.
- The UML diagrams are made for business users, developers, ordinary people, or anyone who is looking forward to understand the system
- Thus it can be concluded that the UML is a simple modeling approach that is used to model all the practical systems.

Characteristics of UML

- The UML has the following features:
  - It is a generalized modeling language.
  - It is distinct from other programming languages like C++, Python, etc.
  - It is interrelated to object-oriented analysis and design.
  - It is used to visualize the workflow of the system.
  - It is a pictorial language, used to generate powerful modeling artifacts.

Cont…

- some object-oriented concepts that are needed to begin with UML:
  - **Object:** An object is a real world entity. There are many objects present within a single system. It is a fundamental building block of UML.
  - **Class:** A class is a software blueprint for objects, which means that it defines the variables and methods common to all the objects of a particular type.
  - **Abstraction:** Abstraction is the process of portraying the essential characteristics of an object to the users while hiding the irrelevant information.
  - **Inheritance:** Inheritance is the process of deriving a new class from the existing ones.
  - **Polymorphism:** It is a mechanism of representing objects having multiple forms used for different purposes.
  - **Encapsulation:** It binds the data and the object together as a single unit, enabling tight coupling between them.

Diagrams in the UML

- The UML diagrams are categorized into
  - **structural diagrams,**
  - **behavioral diagrams,**
  - **interaction diagrams.**
- The diagrams are hierarchically classified in the following figure:

Cont…

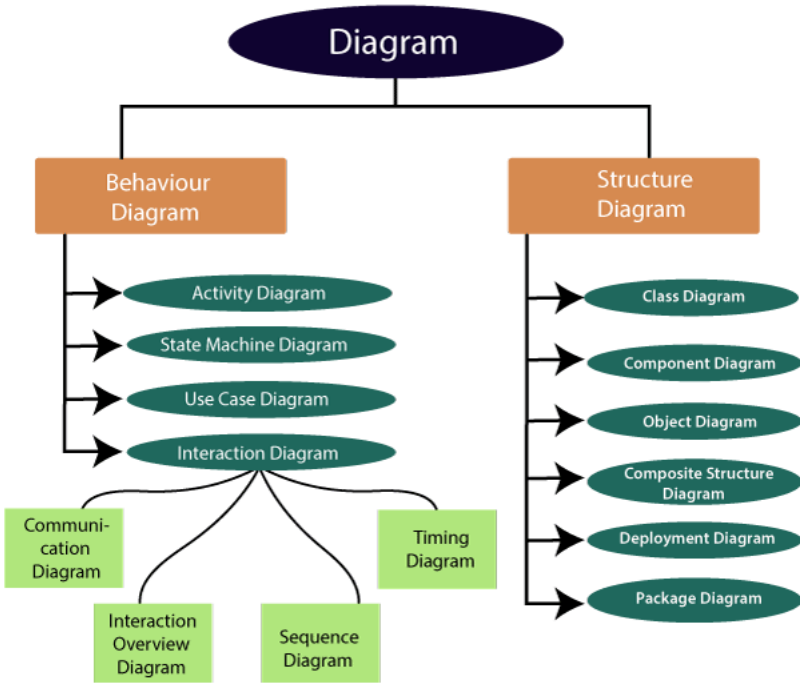


Diagram
Behaviour Diagram
Structure Diagram
Activity Diagram
State Machine Diagram
Use Case Diagram
Interaction Diagram
Communi-cation Diagram
Interaction Overview Diagram
Sequence Diagram
Timing Diagram
Class Diagram
Component Diagram
Object Diagram
Composite Structure Diagram
Deployment Diagram
Package Diagram

UML Diagrams

- **Behavioral Diagrams:** Behavioral diagrams portray a dynamic view of a system or the behavior of a system, which describes the functioning of the system. It includes use case diagrams, state diagrams, and activity diagrams. It defines the interaction within the system.
- **Structural diagrams: it** depict a static view or structure of a system. It is widely used in the documentation of software architecture. It embraces class diagrams, composite structure diagrams, component diagrams, deployment diagrams, object diagrams, and package diagrams.
- **Interaction diagrams** are a subclass of behavioral diagrams that give emphasis to object interactions and also depicts the flow between various use case elements of a system. it shows how objects interact with each other and how the data flows within them. It consists of communication, interaction overview, sequence, and timing diagrams.

System development focuses on three different models of the system.

- **Functional model**, represented in UML with use case diagrams, describes the functionality of the system from the user's point of view.
- The **object model**, represented in UML with class diagrams, describes the structure of the system in terms of objects, attributes, associations, and operations.
- The **dynamic model**, represented in UML with interaction diagrams, state machine diagrams, and activity diagrams, describes the internal behavior of the system.
  - Interaction diagrams describe behavior as a sequence of messages exchanged among a *set of objects*,
  - state machine diagrams describe behavior in terms of states of an *individual object* and the possible transitions between states.
  - Activity diagrams describe behavior in terms control and data flows

UML Use Case Diagram

- A use case diagram is used to represent the dynamic behavior of a system.
- Use cases are used during requirements elicitation and analysis to represent the functionality of the system. Use cases focus on the behavior of the system from an external point of view.
- A **use case diagram** at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved.

Components of use case diagram

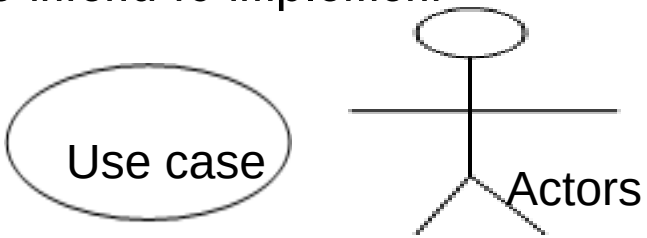
- A **use case** describes a function provided by the system that yields a visible result for an actor.
- An **actor** describes any entity that interacts with the system (e.g., a user, another system, the system's physical environment, role(a system administrator, a bank customer, a bank teller)).
- The identification of actors and use cases results in the definition of the **boundary of the system**, that is, in differentiating the tasks accomplished by the system and the tasks accomplished by its environment.
- The actors are outside the boundary of the system, whereas the use cases are inside the boundary of the system.

Purpose of Use Case Diagrams

- The following are the purposes of a use case diagram given below:
  - The main purpose of a use case diagram is to portray the functionality of the system
  - It gathers the system's needs.
  - It depicts the external view of the system.
  - It recognizes the internal as well as external factors that influence the system.
  - It represents the interaction between the actors.

How to draw a Use Case diagram?

- Rules that must be followed while drawing a use case :
  - An appropriate and meaningful name should be assigned to the actor or a use case of a system.
  - The communication of an actor with a use case must be defined in an understandable way.
  - Specified notations to be used as and when required.
  - The rectangle around the use cases is called the system boundary box, indicates the scope of your system use cases inside the rectangle represent the functionality that we intend to implement
- Diagram representation

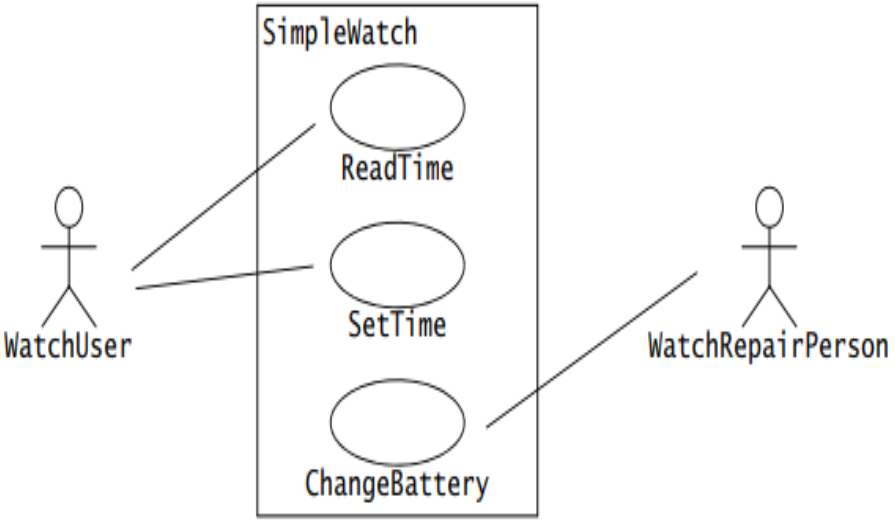


Examples1

- Depicts a use case diagram for a simple watch. The WatchUser actor may either consult the time on their watch (with the ReadTime use case) or set the time (with the SetTime use case). However, only the WatchRepairPerson actor can change the battery of the watch (with the ChangeBattery use case)

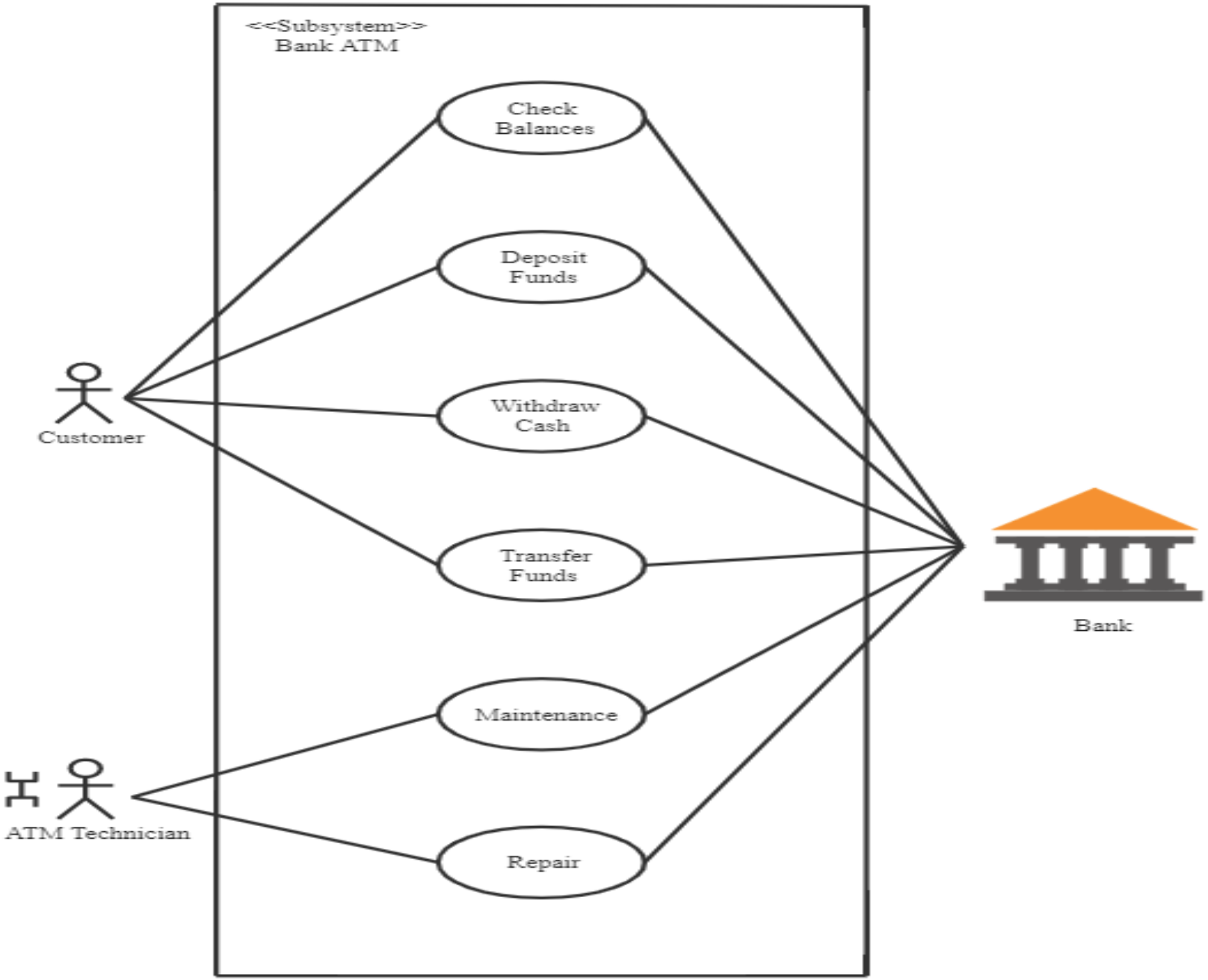
- Actor: WatchUser,
  WatchRepairPerson

- Usecase: ReadTime
  SetTime
  ChangeBattery



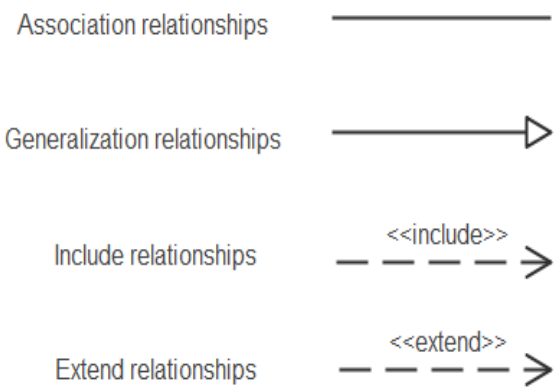
Example 2 Draw Use Case Diagram for ATM?

- **Identify the actors and use case**
- **Actors:**
  - **Customer**
  - **ATM Technician**
  - **Bank**
- **Use case**
  - **Check balance**
  - **Deposit funds**
  - **Withdraw cash**
  - **Transfer funds**
  - **Maintenance**
  - **Repair**


<<Subsystem>>
Bank ATM
Check Balances
Deposit Funds
Withdraw Cash
Transfer Funds
Maintenance
Repair
Customer
ATM Technician
Bank

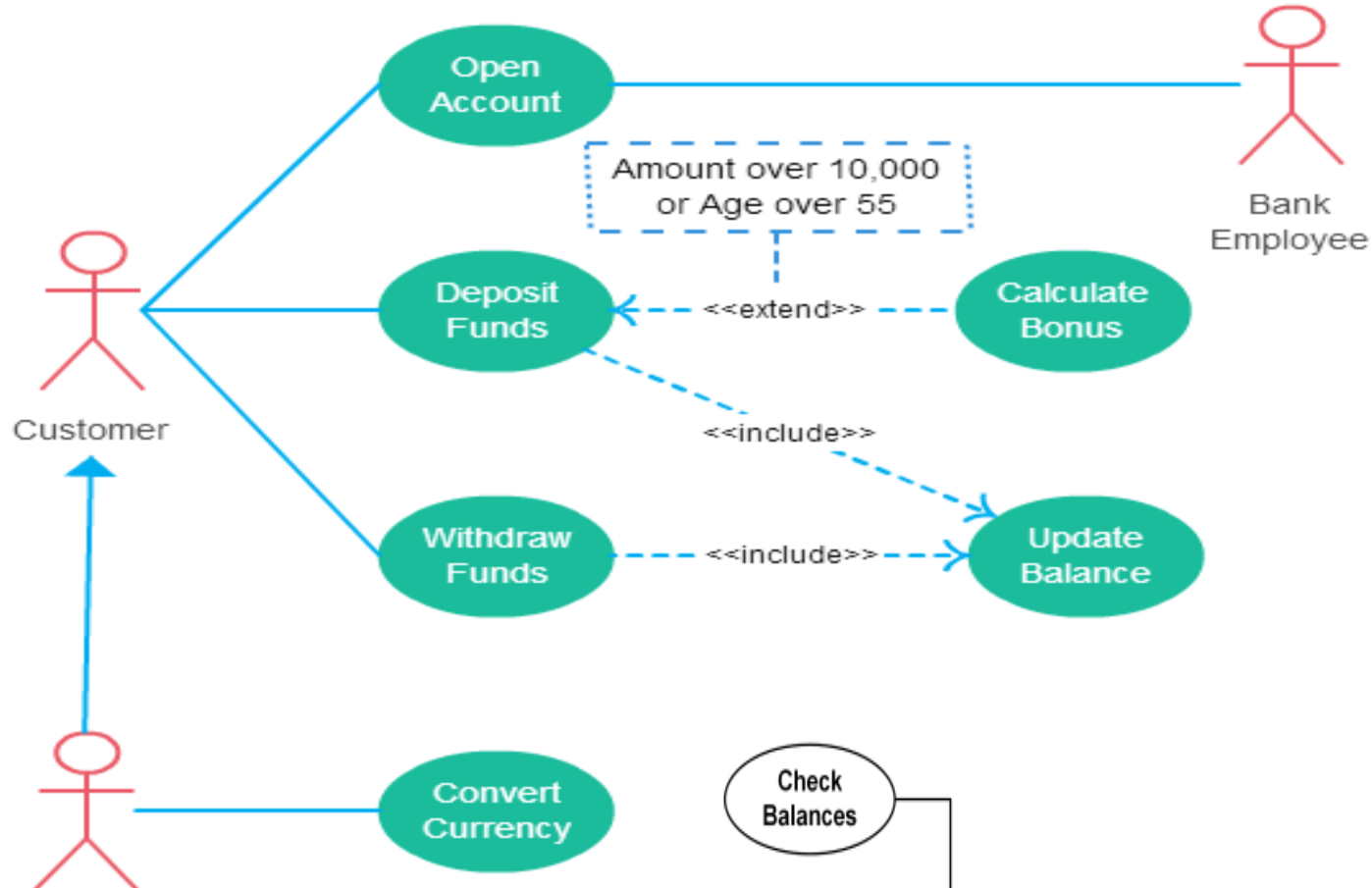
Relationships in Use Case Diagrams

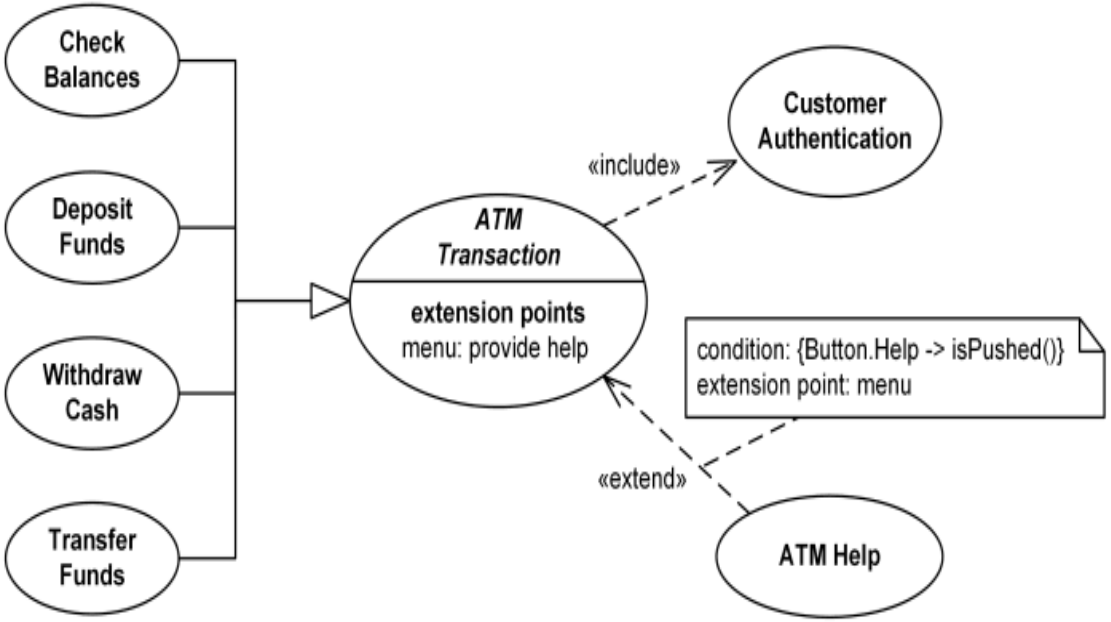
- There are five types of relationships in a use case diagram. They are
- Association between an actor and a use case
- Generalization of an actor: actor can inherit the role of the other actor.
- Extend relationship between two use cases
- Include relationship between two use cases
- Generalization of a use case:
  - The behavior of the ancestor is inherited by the descendant. This is used when there is common behavior between two use cases and also specialized behavior specific to each use case.



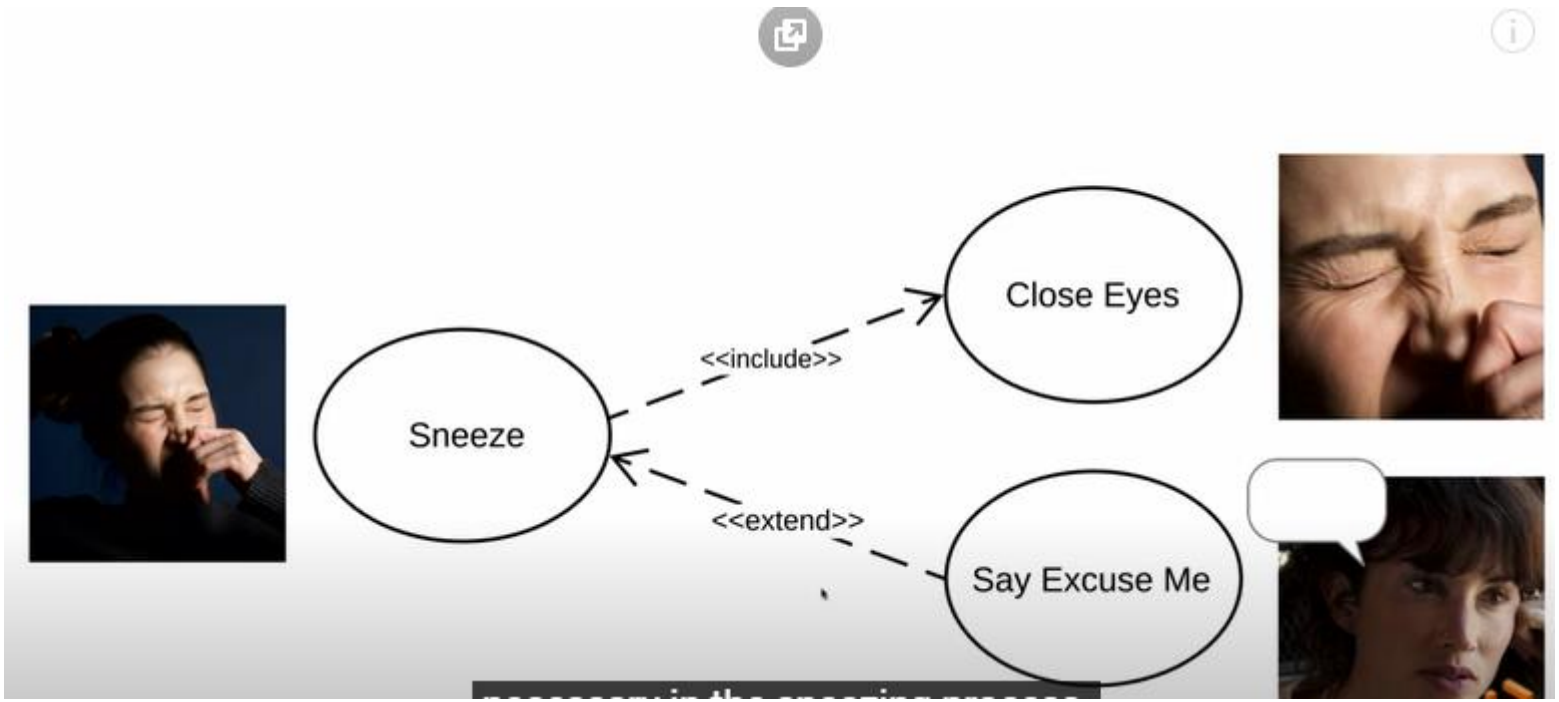
Extend and include relationship

- **Extended :** As the name implies it extends the base use case and adds more functionality to the system. Here are a few things to consider when using the <<**extend**>> relationship.
  - **The extending use case is dependent on the extended (base) use case.** In the below diagram the "Calculate Bonus" use case doesn't make much sense without the "Deposit Funds" use case.
  - **The extending use case is usually optional** and can be triggered conditionally
- **Include:** Include use case is part of the including (base) use case. The main reason for this is to reuse common actions across multiple use cases. In some situations, this is done to simplify complex behaviors
  - The base use case is incomplete without the included use case.
  - The included use case is mandatory and not optional.


Open Account
Bank Employee
Amount over 10,000 or Age over 55
Customer
Deposit Funds
<<extend>>
Calculate Bonus
<<include>>
Withdraw Funds
<<include>>
Update Balance
NRFC Customer
Convert Currency


Check Balances
Deposit Funds
Withdraw Cash
Transfer Funds
ATM Transaction
extension points
menu: provide help
«include»
Customer Authentication
condition: {Button.Help -> isPushed()}
extension point: menu
«extend»
ATM Help

Examples of include and extend



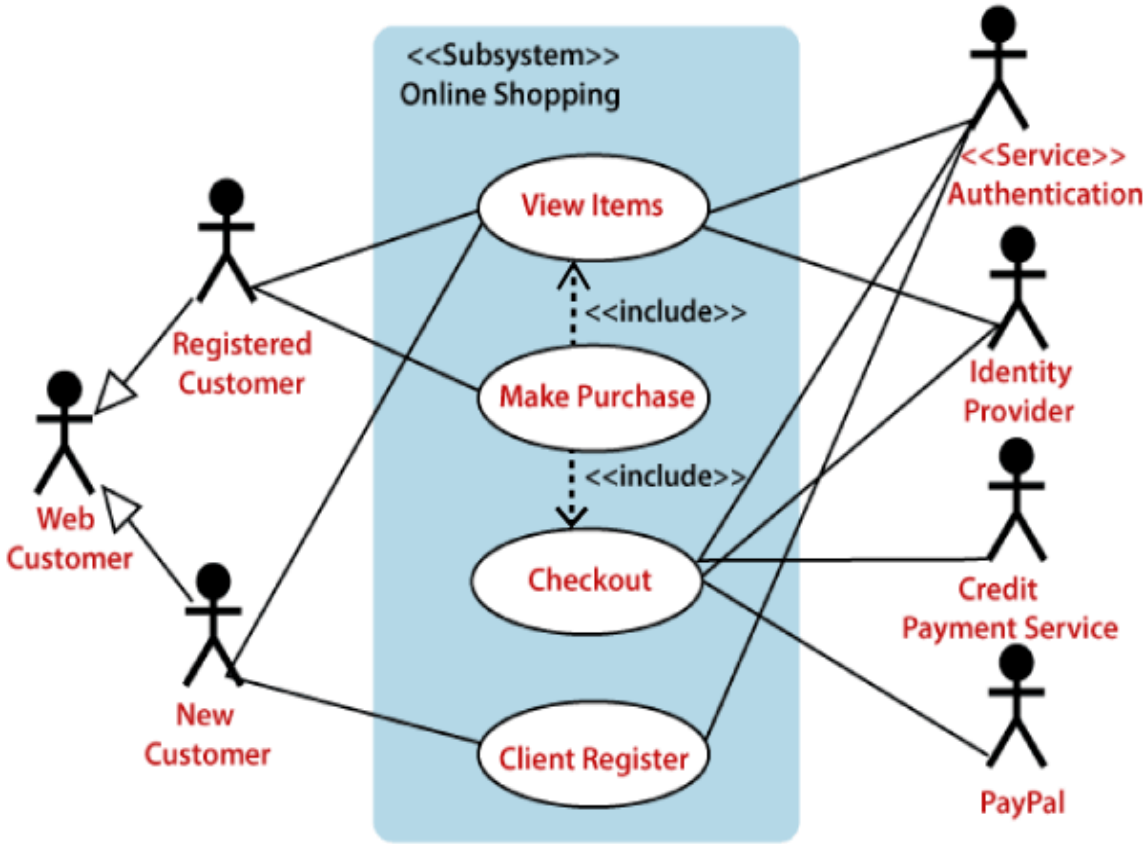
Example of a Use Case Diagram3 Online Shopping website

- A use case diagram depicting the Online Shopping website is given below.
- Here the Web Customer actor makes use of any online shopping website to purchase online. The top-level use case are as follows; View Items, Make Purchase, Checkout, Client Register. The **View Items** use case is utilized by the customer who searches and view products.
- The **Client Register** use case allows the customer to register itself with the website for availing gift vouchers, coupons, or getting a private sale invitation. It is to be noted that the **Checkout** is an included use case, which is part of **Making Purchase**, and it is not available by itself.

Cont…

- Actors: Web Customer, service authentication(admin), finance(credit and PayPal service)…
- Use case : View Items, Make Purchase, Checkout, Client Register.
  - **View Items** is further extended by several use cases such as; Search Items, Browse Items, View Recommended Items, Add to Shopping Cart,

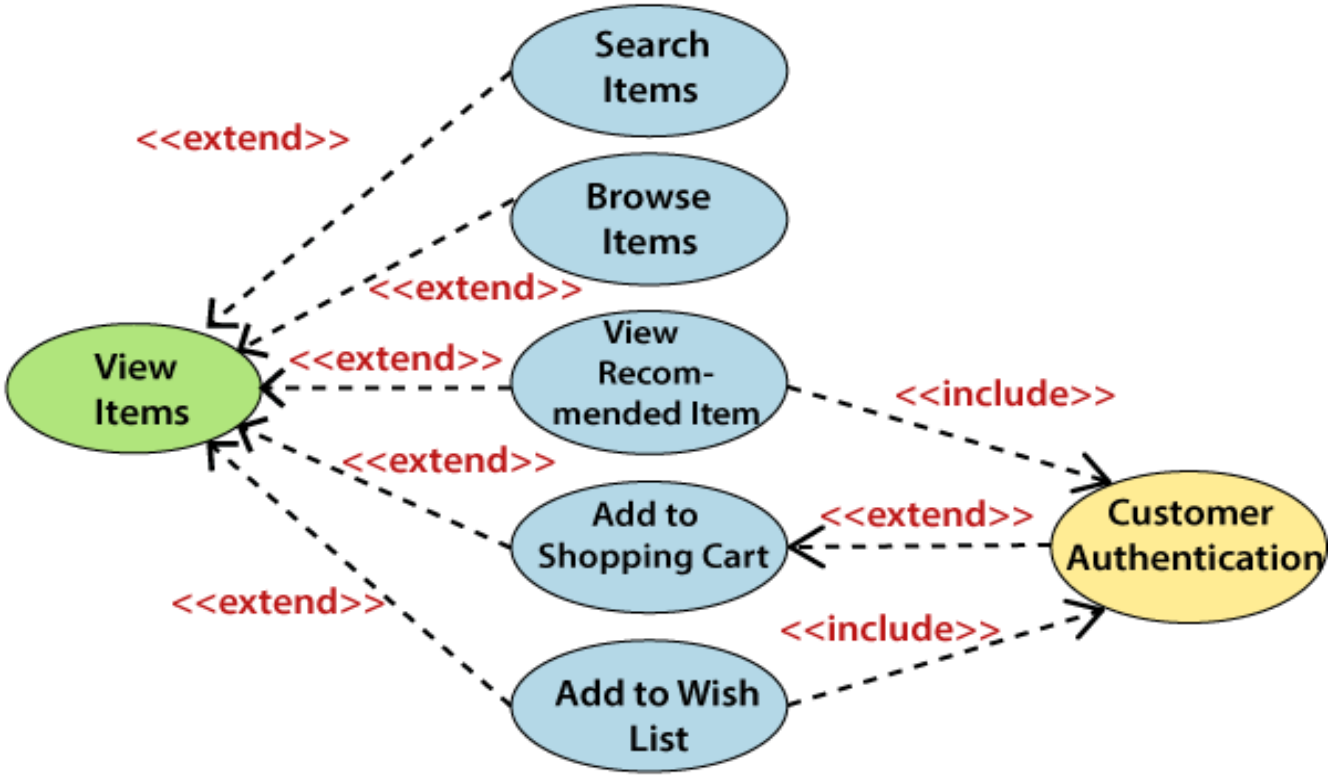
Cont…


<<Subsystem>>
Online Shopping
View Items
<<include>>
Make Purchase
<<include>>
Checkout
Client Register
Registered Customer
Web Customer
New Customer
<<Service>>
Authentication
Identity Provider
Credit Payment Service
PayPal

Cont…

- The **View Items** is further extended by several use cases such as; Search Items, Browse Items, View Recommended Items, Add to Shopping Cart, Add to Wish list. All of these extended use cases provide some functions to customers, which allows them to search for an item.
- Both **View Recommended Item** and **Add to Wish List** include the Customer Authentication use case, as they necessitate authenticated customers, and simultaneously item can be added to the shopping cart without any user authentication.

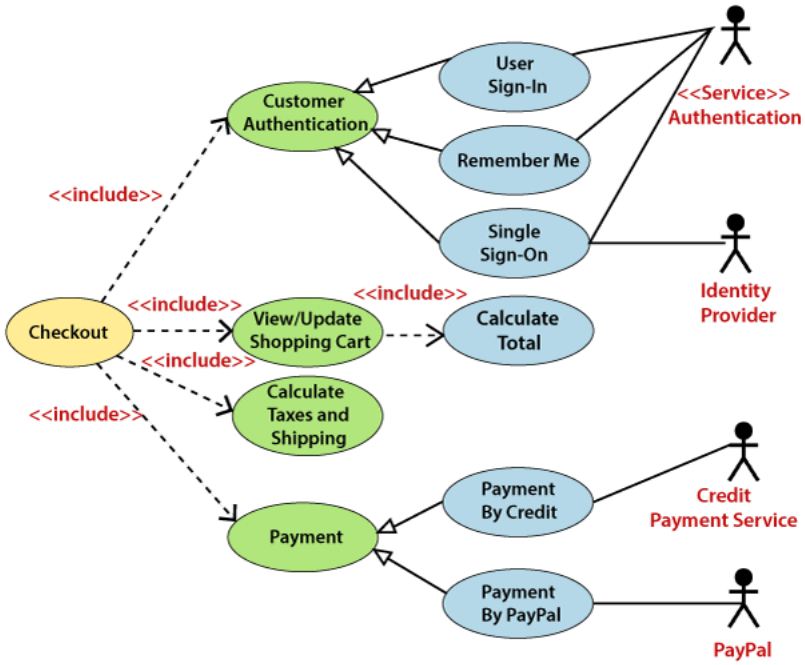
Cont…


Search Items
<<extend>>
Browse Items
<<extend>>
View Items
<<extend>>
View Recom-mended Item
<<include>>
<<extend>>
Add to Shopping Cart
<<extend>>
Customer Authentication
<<extend>>
<<include>>
Add to Wish List

Cont…

- Similarly, the **Checkout** use case also includes the following use cases, as shown below. It requires an authenticated Web Customer, which can be done by login page, user authentication cookie ("Remember me"), or Single Sign-On (SSO). SSO needs an external identity provider's participation, while Web site authentication service is utilized in all these use cases.
- The Checkout use case involves Payment use case that can be done either by the credit card and external credit payment services or with PayPal.
-

Cont…


User
Sign-In
Customer
Authentication
Remember Me
<<Service>>
Authentication
<<include>>
Single
Sign-On
Identity
Provider
<<include>>
<<include>>
Checkout
View/Update
Shopping Cart
Calculate
Total
<<include>>
Calculate
Taxes and
Shipping
<<include>>
Payment
By Credit
Credit
Payment Service
Payment
Payment
By PayPal
PayPal

Important tips for drawing a Use Case diagram

- Following are some important tips that are to be kept in mind while drawing a use case diagram:
  - A simple and complete use case diagram should be articulated.
  - A use case diagram should represent the most significant interaction among the multiple interactions.
  - At least one module of a system should be represented by the use case diagram.
  - If the use case diagram is large and more complex, then it should be drawn more generalized.

UML Class Diagram

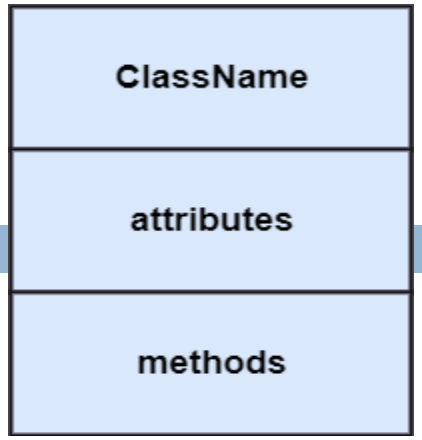
- Class diagrams are used to describe the structure of the system. Classes are abstractions that specify the common structure and behavior of a set of objects.
- Class diagrams describe the system in terms of objects, classes, attributes, operations, and their associations to give an overview of the software system.
- The class diagram depicts a static view of an application. A class consists of its objects, and also it may inherit from other classes.

Purpose of Class Diagrams

- It is the only diagram that is widely used for construction, and it can be mapped with object-oriented languages. It is the most popular UML diagrams.
- Following are the purpose of class diagrams given below:
  - It analyses and designs a static view of an application.
  - It describes the major responsibilities of a system.
  - It is a base for component and deployment diagrams.
  - It incorporates forward and reverse engineering.
  - . It represents the mapping with object-oriented languages that are C++, Java, etc.

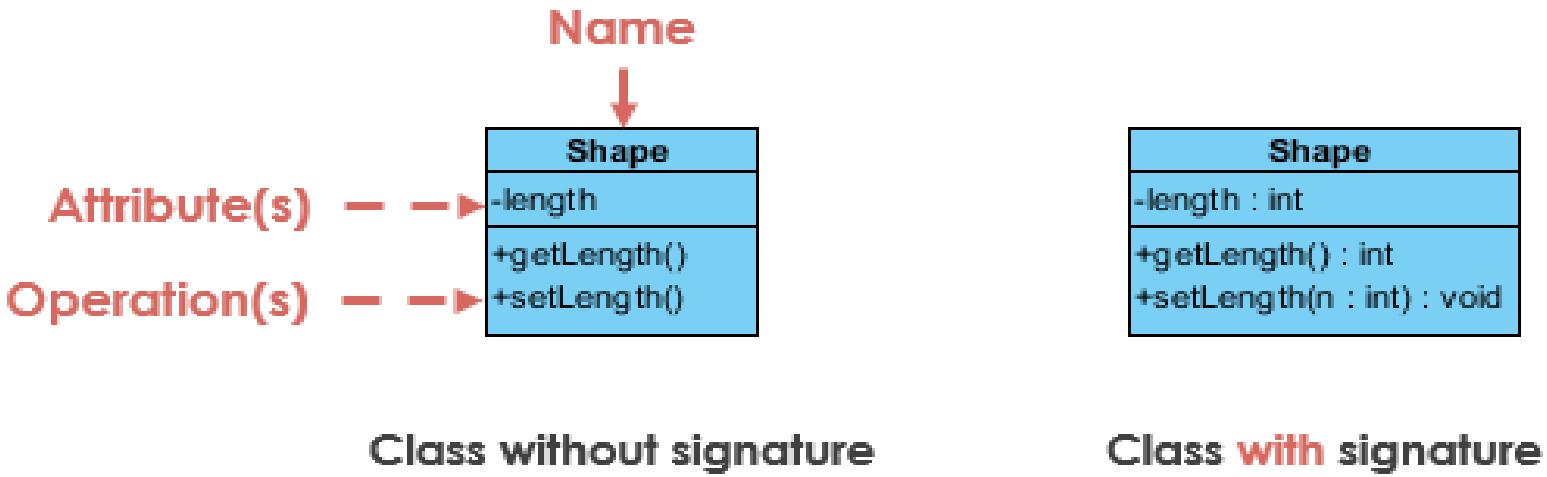
Vital components of a Class Diagram

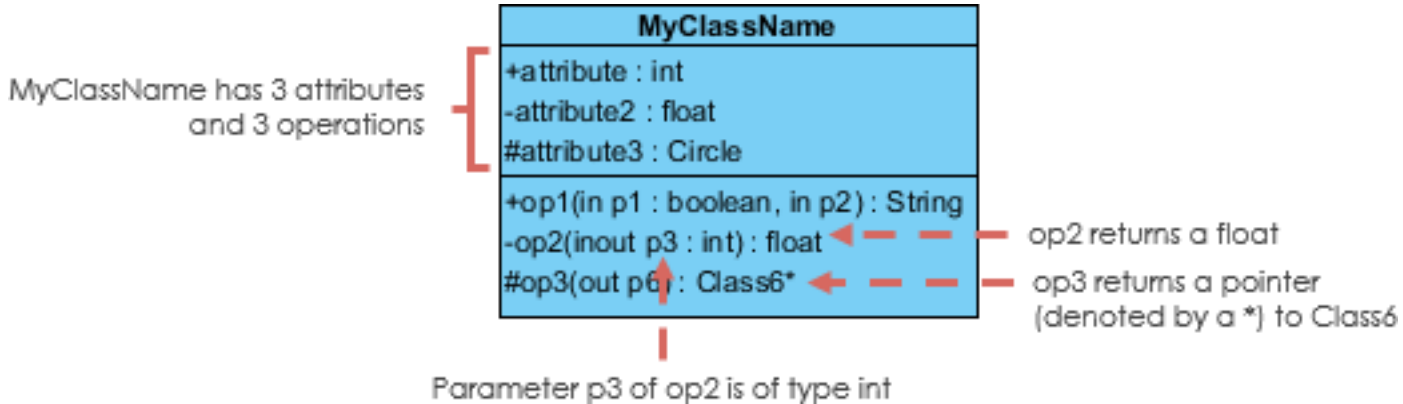
- The class diagram is made up of three sections:
- **Upper Section:** The upper section encompasses the name of the class. A class is a representation of similar objects that shares the same relationships, attributes, operations, and semantics.
- Some of the following rules that should be taken into account while representing a class are given below:
  - Capitalize the initial letter of the class name.
  - Place the class name in the center of the upper section.
  - A class name must be written in bold format.



- **Middle Section:** The middle section constitutes the attributes, which describe the quality of the class. The attributes have the following characteristics:
  - The attributes are written along with its visibility factors, which are public (+), private (-), protected (#), and package (~).
  - The accessibility of an attribute class is illustrated by the visibility factors.
  - A meaningful name should be assigned to the attribute, which will explain its usage inside the class.
- **Lower Section:** The lower section contain methods or operations. The methods are represented in the form of a list, where each method is written in a single line. It demonstrates how a class interacts with data.

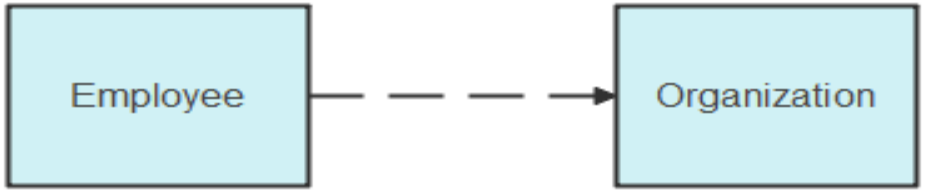
Simple example class diagram





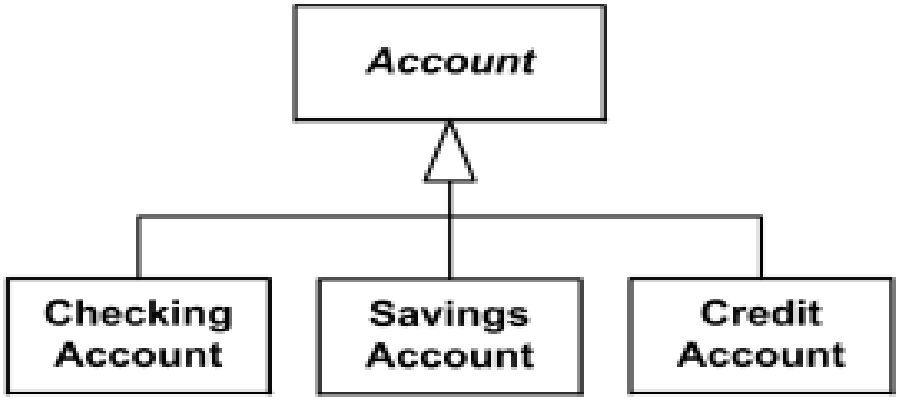
Relationships

- In UML, relationships are of three types:
- **Dependency:** A dependency is a semantic relationship between two or more classes where a change in one class cause changes in another class. It forms a weaker relationship.
- For example, an employee is dependent on the organization..



Cont…

- **Generalization:** A generalization is a relationship between a parent class (superclass) and a child class (subclass). In this, the child class is inherited from the parent class.
- For example, The Checking Account, Saving Account, and Credit Account are the generalized form of Bank Account.

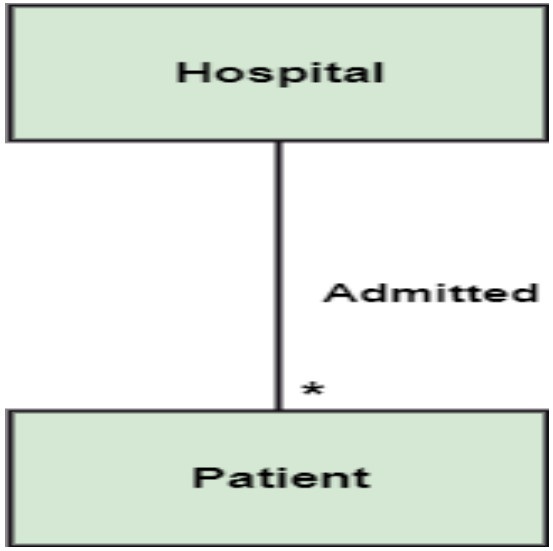


Cont…

- **Association:** It describes a static or physical connection between two or more objects. It depicts how many objects are there in the relationship.
  For example, a department is associated with the college.

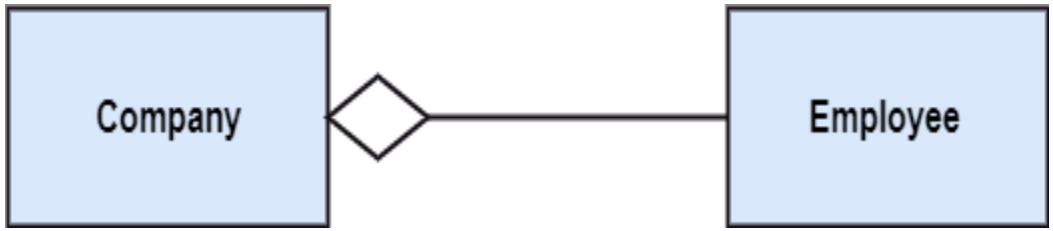
Cont…

- **Multiplicity:** It defines a specific range of allowable instances of attributes. In case if a range is not specified, one is considered as a default multiplicity.
- For example, multiple patients are admitted to one hospital.



Cont…

- **Aggregation:** An aggregation is a subset of association, which represents has a relationship. It is more specific then association.
- It defines a part-whole or part-of relationship. In this kind of relationship, the child class can exist independently of its parent class.
- The company encompasses a number of employees, and even if one employee resigns, the company still exists.

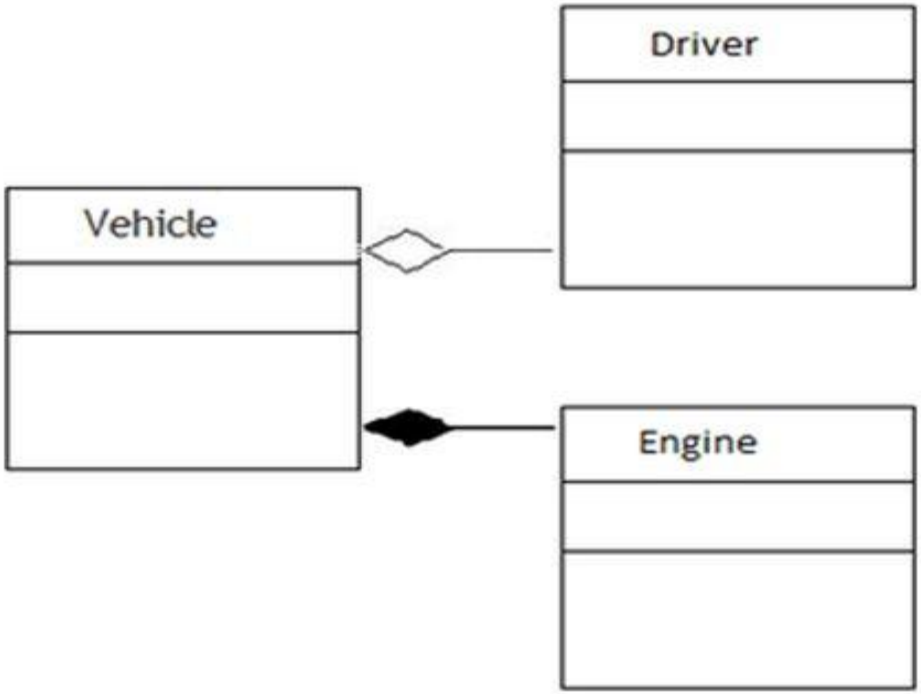


Cont…

- **Composition:** The composition is a subset of aggregation. It portrays the dependency between the parent and its child, which means if one part is deleted, then the other part also gets discarded. It represents a whole-part relationship.
- A contact book consists of multiple contacts, and if you delete the contact book, all the contacts will be lost.
- Composition is special type of Aggregation which is the child object does not have their own life cycle.

Cont…

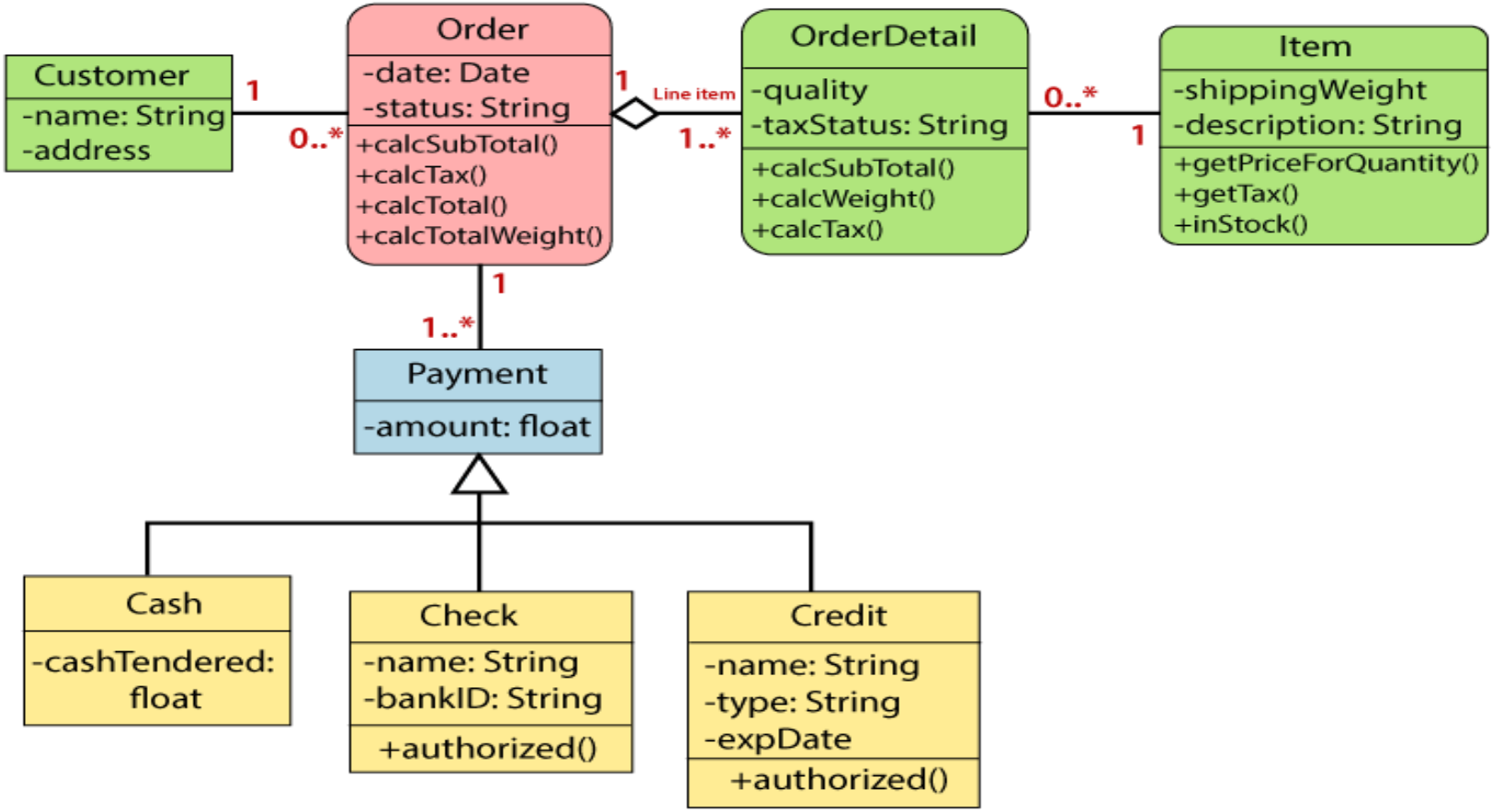
- Combined example of Aggregation and Composition. vehicle can exist without driver. But vehicle can not exist without Engine.



How to draw a Class Diagram?

- A collection of class diagrams as a whole represents a system. Some key points to keep in mind while drawing a class diagram are:
  1. To describe a complete aspect of the system, it is suggested to give a meaningful name to the class diagram.
  2. The objects and their relationships should be acknowledged in advance.
  3. The attributes and methods of each class must be known.
  4. A minimum number of desired properties should be specified as more number of the unwanted property will lead to a complex diagram.
  5. Notes can be used as and when required by the developer to describe the aspects of a diagram.
  6. The diagrams should be redrawn and reworked as many times to make it correct before producing its final version.

Class Diagram Example: A class diagram describing the sales order system is given below.

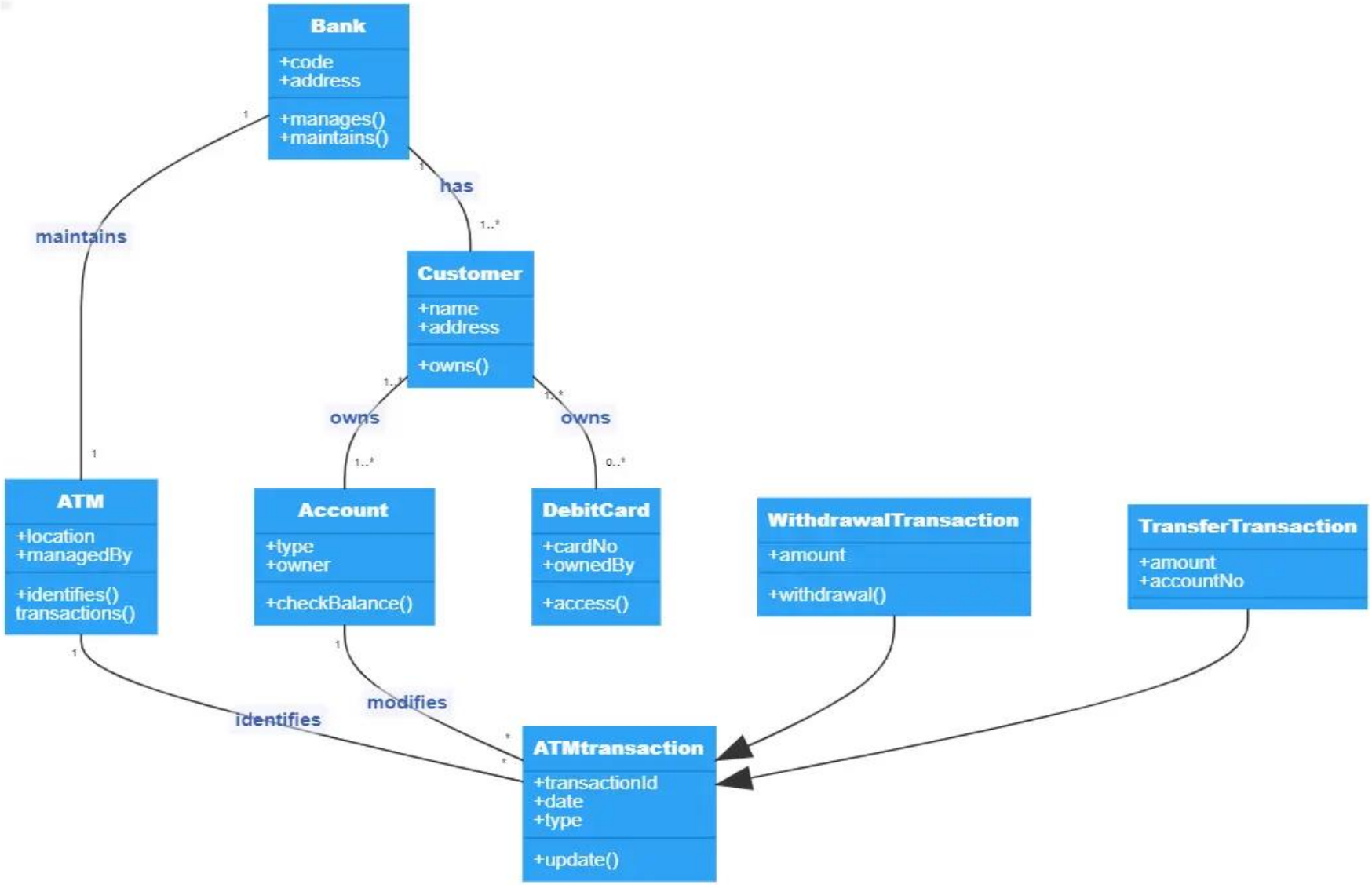


Example2 class diagram for ATM system

- The bank class needs a code to identify it and an address. The methods that the bank class can use to interact with its ATM are manage and maintain.
- An ATM is managed by the bank and it has a location. Its methods are related to transactions, which it identifies and carries out. The label “maintains” explains the relationship between the bank and the ATM, while a bank may or may not have more many ATMs.
- Every bank needs customers, Customers need a name and an address. The relationship between the bank and the customer is that the bank has customers. The only method for the customer is to own an account and to own a debit card.
- An account is owned by a customer and also has a type, such as savings or checking. It has a method to enable it to check the account balance.

Cont…

- The DebitCard class has a card number and is also owned by a customer. Its method enables it to access the ATM and therefore the account. Now we need to model the transactions that can take place when the customer uses the ATM. The base class for this is ATMTransaction. It has three attributes: transaction ID, date, and type. Its method is to update. Let's imagine that the customer can perform two different types of transactions. They can withdraw funds or they can transfer funds to another account.
- a WithdrawalTransaction class has an amount attribute and a withdrawal method. a TransferTransaction class also has an amount attribute, but in addition it has an account number attribute so that the customer can send the money to a different account.
- These two transaction are share the ATMTransaction class as their parent class. For instance, these transactions can make use of the update method to interact with the account class.


Bank
+code
+address
+manages()
+maintains()
maintains
has
1
1..*
Customer
+name
+address
+owns()
owns
owns
1..*
1..*
1..*
0..*
ATM
+location
+managedBy
+identifies()
transactions()
Account
+type
+owner
+checkBalance()
DebitCard
+cardNo
+ownedBy
+access()
WithdrawalTransaction
+amount
+withdrawal()
TransferTransaction
+amount
+accountNo
identifies
modifies
ATMtransaction
+transactionId
+date
+type
+update()

Sequence Diagram

- The sequence diagram its an interaction diagram that shows the dynamic models of the system, which represents the flow of messages in the system and is also termed as an event diagram.
- It is an Interaction diagrams describe behavior as a sequence of messages exchanged among a *set of objects*,
- It portrays the communication between any two lifelines as a time-ordered sequence of events, such that these lifelines took part at the run time.
- It shows the interactions between the objects in terms of messages exchanged over time. It describes in what order and how the object functions are in a system.

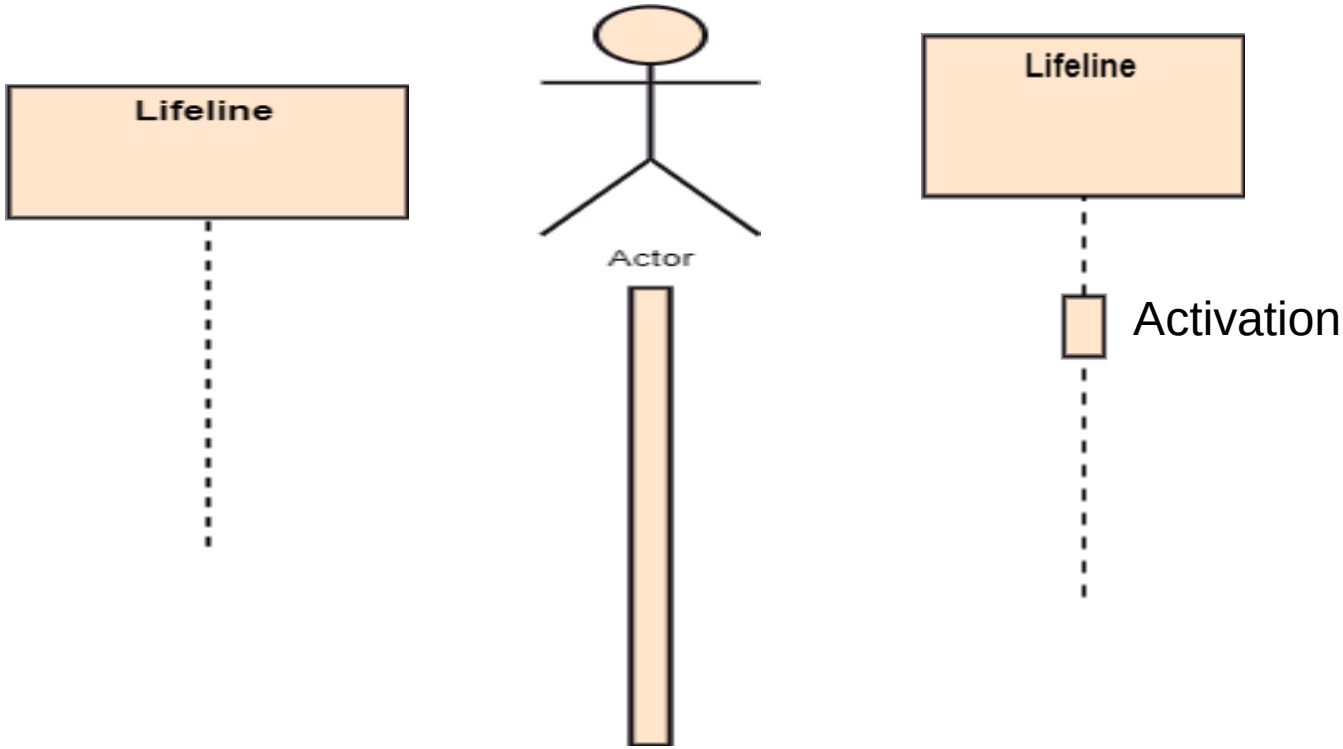
Purpose of a Sequence Diagram

- To model high-level interaction among active objects within a system.
- To model interaction among objects inside a collaboration realizing a use case.
- It either models generic interactions or some certain instances of interaction.

Notations of a Sequence Diagram

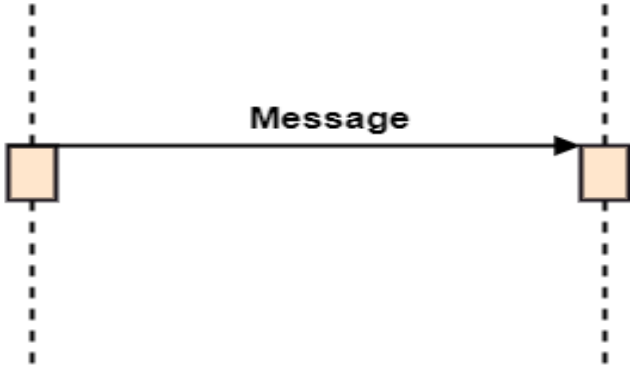
- **Lifeline:** An individual participant in the sequence diagram is represented by a lifeline. It is positioned at the top of the diagram.
- **Actor:** A role played by an entity that interacts with the subject is called as an actor. It is out of the scope of the system. It represents the role, which involves human users and external hardware or subjects. An actor may or may not represent a physical entity, but it purely depicts the role of an entity.
- **Activation:** It is represented by a thin rectangle on the lifeline. It describes that time period in which an operation is performed by an element, such that the top and the bottom of the rectangle is associated with the initiation and the completion time, each respectively.

Cont…



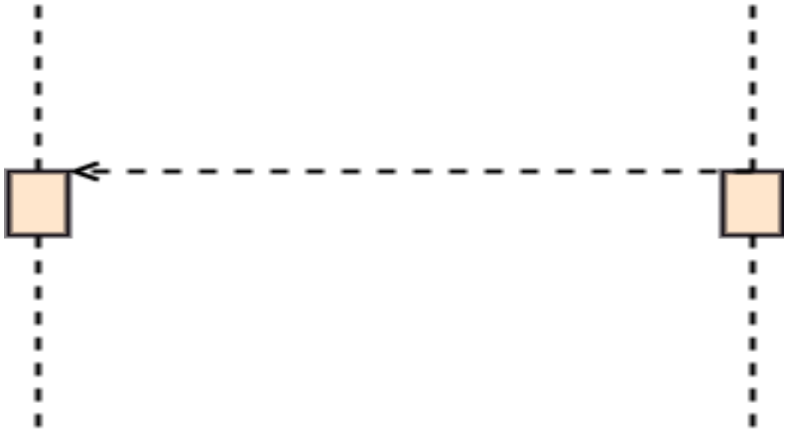
Messages

- The messages depict the interaction between the objects and are represented by arrows. They are in the sequential order on the lifeline. The core of the sequence diagram is formed by messages and lifelines.
- **Call Message:** It defines a particular communication between the lifelines of an interaction, which represents that the target lifeline has invoked an operation.



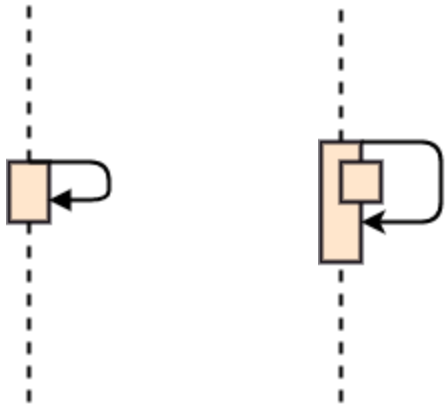
Cont…

- **Return Message:** It defines a particular communication between the lifelines of interaction that represent the flow of information from the receiver of the corresponding caller message.



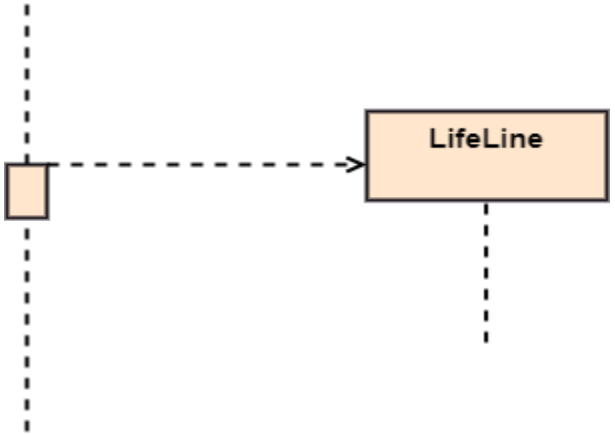
Cont…

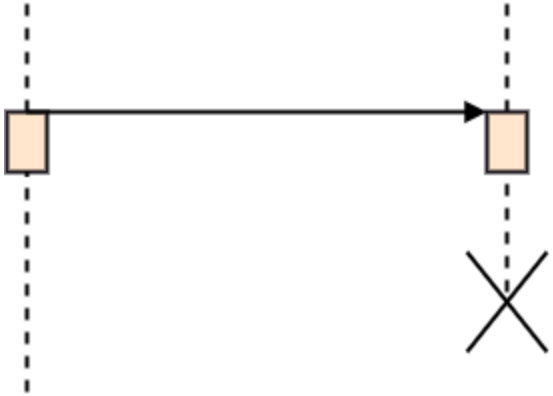
- **Self Message:** It describes a communication, particularly between the lifelines of an interaction that represents a message of the same lifeline, has been invoked.
- **Recursive Message:** A self message sent for recursive purpose is called a recursive message. In other words, it can be said that the recursive message is a special case of the self message as it represents the recursive calls.



Cont…

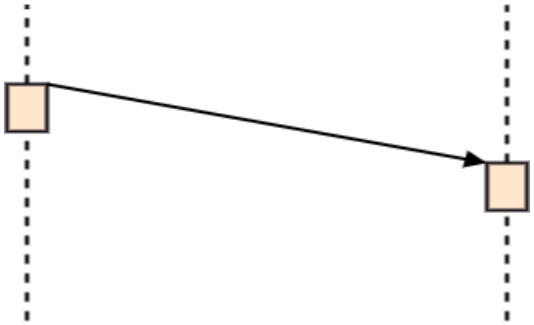
- **Create Message:** It describes a communication, particularly between the lifelines of an interaction describing that the target (lifeline) has been instantiated.
- **Destroy Message:** It describes a communication, particularly between the lifelines of an interaction that depicts a request to destroy the lifecycle of the target.

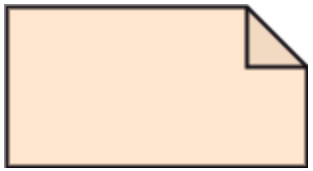




Cont..

- **Duration Message:** It describes a communication particularly between the lifelines of an interaction, which portrays the time passage of the message while modeling a system.
- **Note:** A note is the capability of attaching several remarks to the element. It basically carries useful information for the modelers.

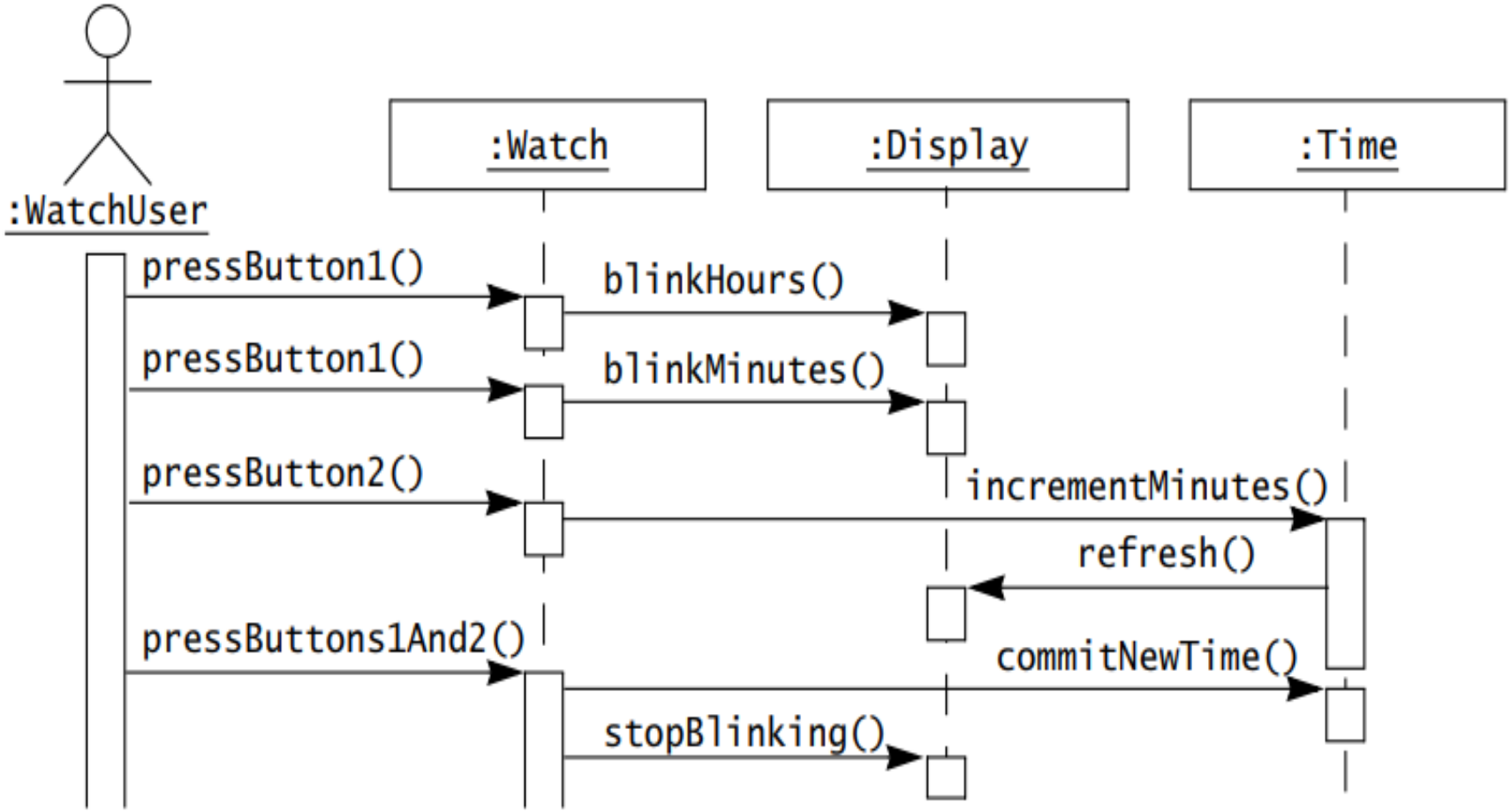




Example 1 Sequence Diagram for simple-watch system,

- Next slide is a special form of interaction diagram, called a **sequence diagram**, for the **SetTime** use case of our simple watch.
- The left-most column represents the WatchUser actor who initiates the use case. Labeled arrows represent stimuli that an actor or an object sends to other objects.
- In this case, the WatchUser presses button 1 twice and button 2 once to set her watch a minute ahead.
- The SetTime use case terminates when the WatchUser presses both buttons simultaneously.

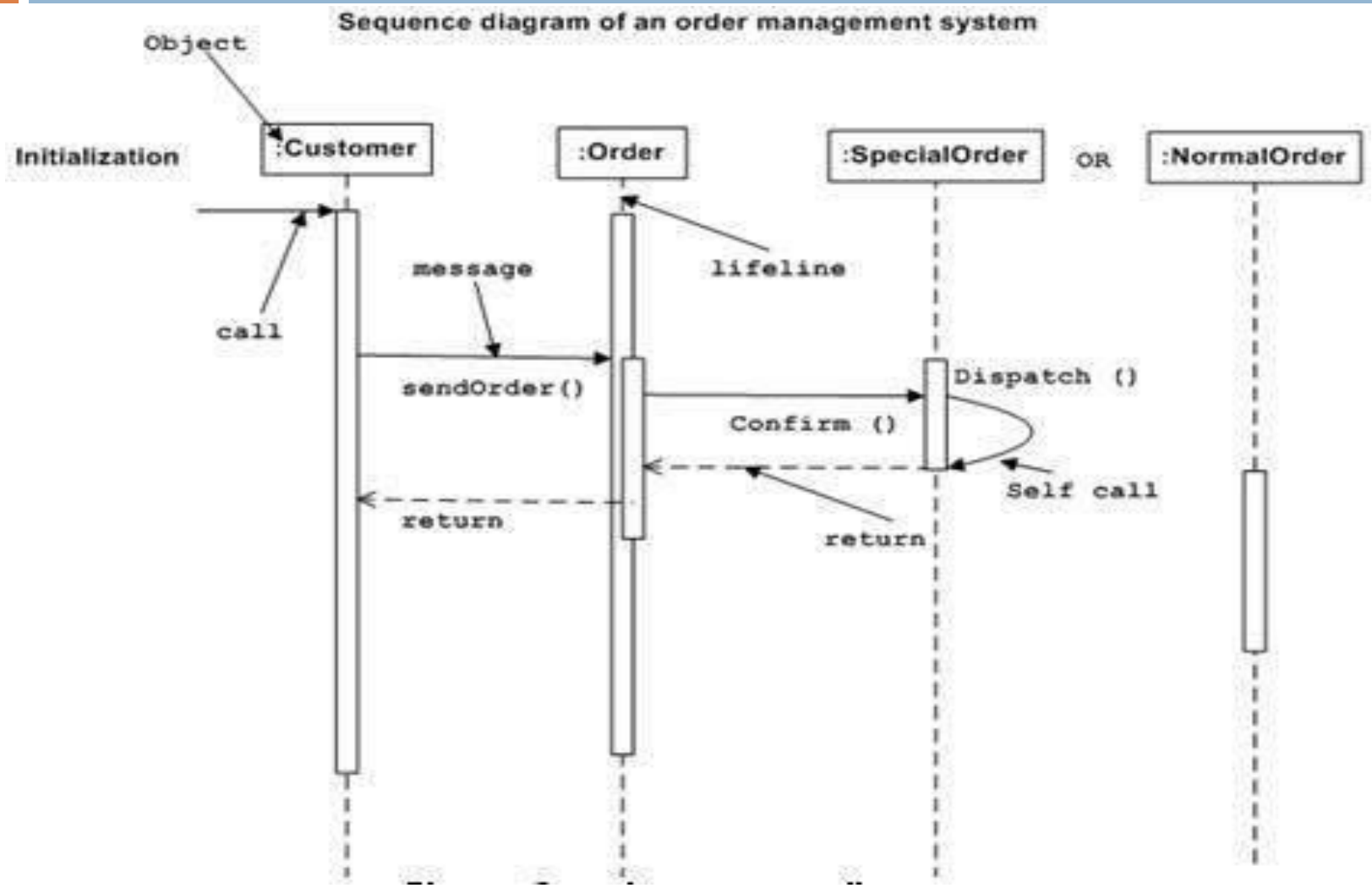
Cont…


:WatchUser
:Watch
:Display
:Time
pressButton1()
blinkHours()
pressButton1()
blinkMinutes()
pressButton2()
incrementMinutes()
refresh()
pressButtons1And2()
commitNewTime()
stopBlinking()

Example of a Sequence Diagram order management system

- The sequence diagram has four objects (Customer, Order, SpecialOrder and NormalOrder).
- The following diagram shows the message sequence for *SpecialOrder* object and the same can be used in case of *NormalOrder* object. It is important to understand the time sequence of message flows.
- The first call is *sendOrder ()* which is a method of *Order object*. The next call is *confirm ()* which is a method of *SpecialOrder* object and the last call is *Dispatch ()* which is a method of *SpecialOrder* object.
- This is also the actual scenario when the system is running.

Cont…


Sequence diagram of an order management system
Object
Initialization
:Customer
:Order
:SpecialOrder
OR
:NormalOrder
call
message
lifeline
sendOrder()
Dispatch ()
Confirm ()
Self call
return
return

Cont…

- The first call is *sendOrder ()* which is a method of *Order object*. The next call is *confirm ()* which is a method of *SpecialOrder* object and the last call is *Dispatch ()* which is a method of *SpecialOrder* object.
- **Benefits of a Sequence Diagram**
  - It explores the real-time application.
  - It depicts the message flow between the different objects.
  - It has easy maintenance.
  - Implement both forward and reverse engineering.
  - It can easily update as per the new change in the system

The drawback of a Sequence Diagram

- In the case of too many lifelines, the sequence diagram can get more complex.
- The incorrect result may be produced, if the order of the flow of messages changes.
- Since each sequence needs distinct notations for its representation, it may make the diagram more complex.
- The type of sequence is decided by the type of message.

UML Activity Diagram

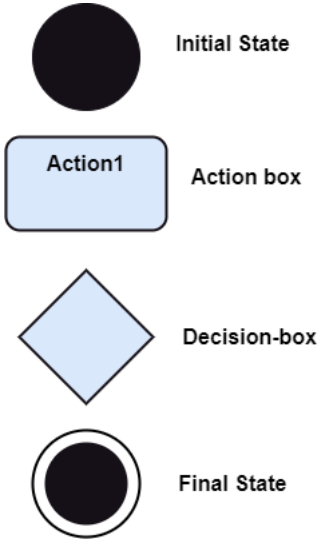
- An activity diagram describes the behavior of a system in terms of activities. Activities are modeling elements that represent the execution of a set of operations.
- The execution of an activity can be triggered by the completion of other activities, by the availability of objects, or by external events.
- Activity **diagrams are similar to flowchart diagrams in that they can be used to represent control flow** (i.e., the order in which operations occur) and data flow (i.e., the objects that are exchanged among operations).
- Activity diagrams describe behavior in terms control and data flows. The activity diagram helps in envisioning the workflow from one activity to another..

Notation of an Activity diagram

Activity diagram constitutes following notations:

- **Initial State:** It depicts the initial stage or beginning of the set of actions.
- **Final State:** It is the stage where all the control flows and object flows end.
- **Decision Box:** It makes sure that the control flow or object flow will follow only one path.
- **Action Box:** It represents the set of actions that are to be performed.

Cont…



Why use Activity Diagram?

- An event is created as an activity diagram encompassing a group of nodes associated with edges. To model the behavior of activities, they can be attached to any modeling element.
- It mainly models processes and workflows. It envisions the dynamic behavior of the system as well as constructs a runnable system that incorporates forward and reverse engineering.
- It is the same as that of a flowchart but not exactly a flowchart itself. It is used to depict the flow between several activities.

How to draw an Activity Diagram?

- An activity diagram is a flowchart of activities, as it represents the workflow among various activities. They are identical to the flowcharts, but they them self are not exactly the flowchart.
- In other words, it can be said that an activity diagram is an enhancement of the flowchart,
- Since it incorporates swim lanes, branching, parallel flows, join nodes, control nodes, and forks, it supports exception handling.
- All of the activities are explored after they are properly analyzed for finding out the constraints applied to the activities.
- Each and every activity, condition, and association must be recognized.

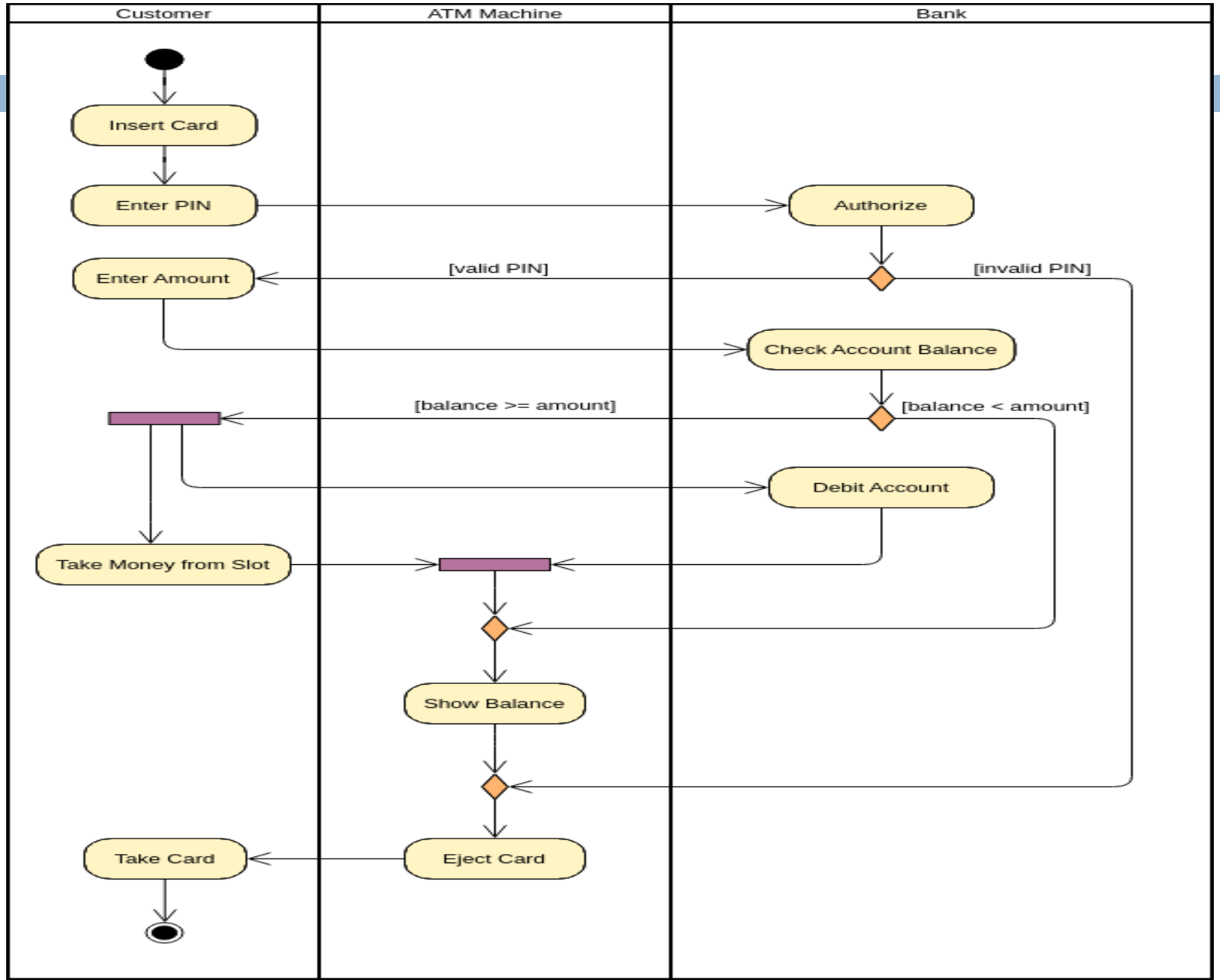
Cont…

- Following are the rules that are to be followed for drawing an activity diagram:
  - A meaningful name should be given to each and every activity.
  - Identify all of the constraints.
  - Acknowledge the activity associations.

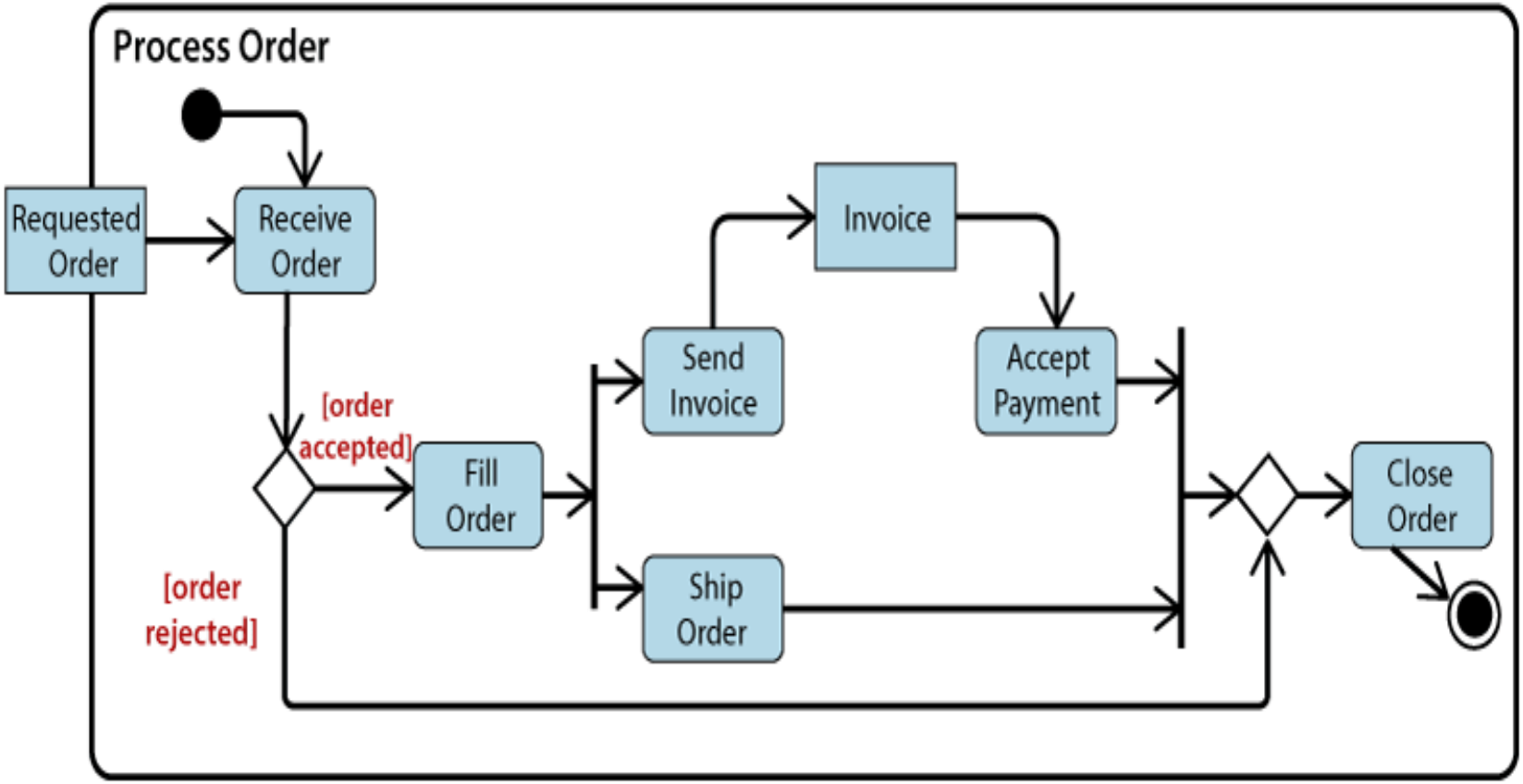
Example of an Activity Diagram

- An example of an activity diagram showing the business flow activity of order processing is given below.
- Here the input parameter is the Requested order, and once the order is accepted, all of the required information is then filled, payment is also accepted, and then the order is shipped. It permits order shipment before an invoice is sent or payment is completed.

Activity diagram ATM System

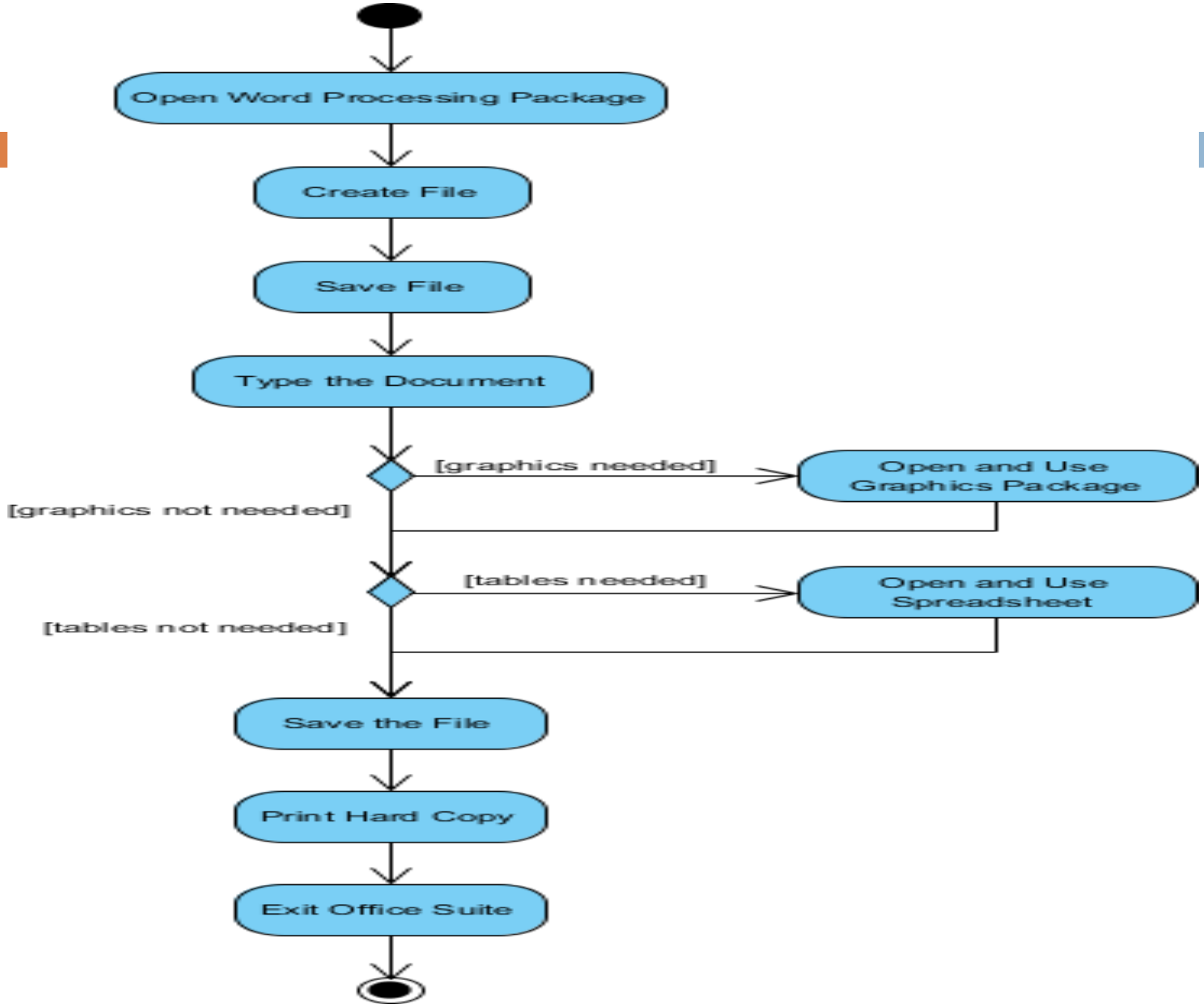


Cont..


Process Order
Requested Order
Receive Order
[order accepted]
[order rejected]
Fill Order
Send Invoice
Invoice
Accept Payment
Ship Order
Close Order

Example 3 Activity Diagram - Modeling a Word Processor

- The activity diagram example below describes the workflow for a word process to create a document through the following steps:
  - Open the word processing package.
  - Create a file.
  - Save the file under a unique name within its directory.
  - Type the document.
  - If graphics are necessary, open the graphics package, create the graphics, and paste the graphics into the document.
  - If a spreadsheet is necessary, open the spreadsheet package, create the spreadsheet, and paste the spreadsheet into the document.
  - Save the file.
  - Print a hard copy of the document.
  - Exit the word processing package.


Open Word Processing Package
Create File
Save File
Type the Document
[graphics needed]
Open and Use Graphics Package
[graphics not needed]
[tables needed]
Open and Use Spreadsheet
[tables not needed]
Save the File
Print Hard Copy
Exit Office Suite

When to use an Activity Diagram?

- An activity diagram can be used to portray business processes and workflows. Also, it used for modeling business as well as the software. An activity diagram is utilized for the followings:
  - To graphically model the workflow in an easier and understandable way.
  - To model the execution flow among several activities.
  - To model comprehensive information of a function or an algorithm employed within the system.
  - To model the business process and its workflow.
  - To generate the top-level flowcharts for representing the workflow of an application.
  - To represent a high-level view of a distributed or an object-oriented system.

UML Deployment Diagram

- The deployment diagram visualizes the physical hardware on **which the software will be deployed**.
- It portrays the static deployment view of a system. It involves the nodes and their relationships.
- It ascertains how software is deployed on the hardware. It maps the software architecture created in design to the physical system architecture, where the software will be executed as a node.
- Since it involves many nodes, the relationship is shown by utilizing communication paths

Purpose of Deployment Diagram

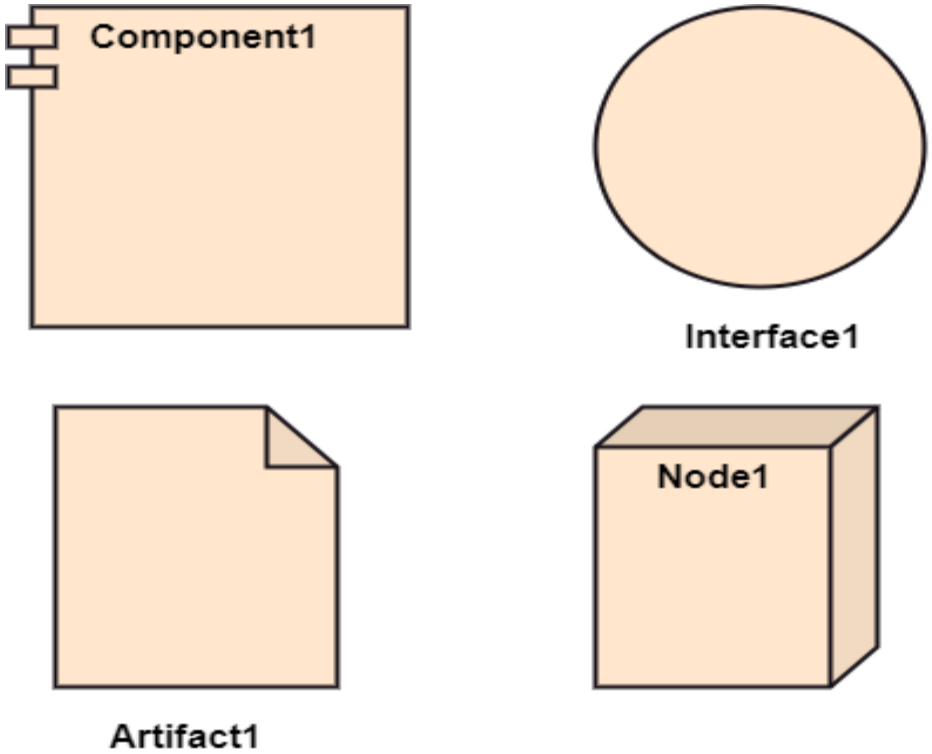
- The main purpose of the deployment diagram is to represent how software is installed on the hardware component.
- It depicts in what manner a software interacts with hardware to perform its execution.
- Both the deployment diagram and the component diagram are closely interrelated to each other as they focus on software and hardware components.
- The component diagram represents the components of a system, whereas the deployment diagram describes how they are actually deployed on the hardware.

Cont…

- The deployment diagram does not focus on the logical components of the system, but it put its attention on the hardware topology.
- Following are the purposes of deployment diagram enlisted below:
  - To envision the hardware topology of the system.
  - To represent the hardware components on which the software components are installed.
  - To describe the processing of nodes at the runtime.

Symbol and notation of Deployment diagram

- The deployment diagram consist of the following notations:
  - A component
  - An artifact(files)
  - An interface
  - A node(HW or SW)



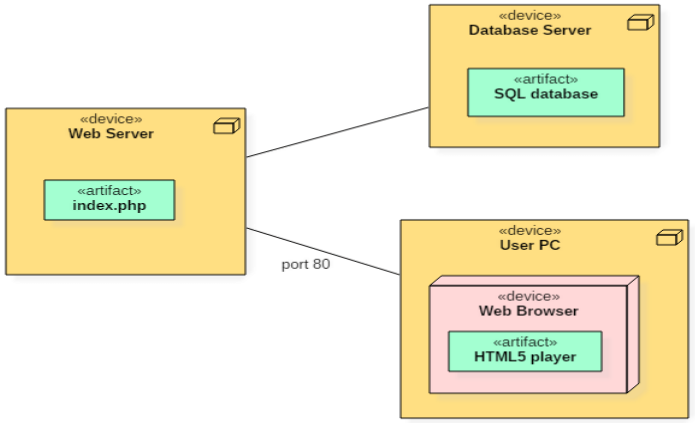
How to draw a Deployment Diagram?

- The nodes are used to execute the artifacts. The instances of artifacts can be deployed on the instances of nodes.
- Since it plays a critical role during the administrative process, it involves the following parameters:
  - High performance
  - Scalability
  - Maintainability
  - Portability
  - Easily understandable

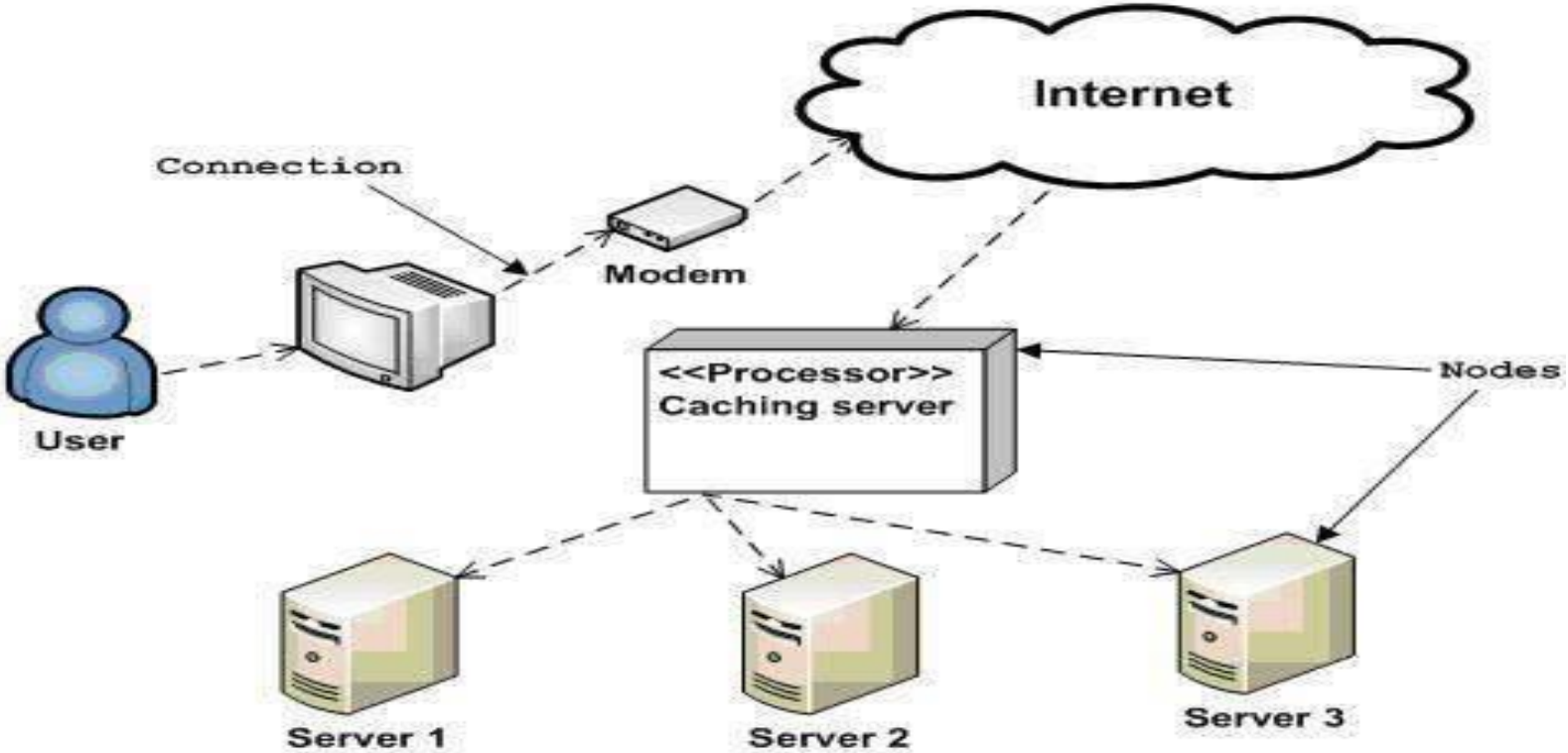
Example of a Deployment diagram

- Before drawing a deployment diagram, the following artifacts should be identified –
  - **Nodes** and **Relationships among nodes**
- The Following is a sample deployment diagram to provide an idea of the deployment view of order management system. Here, we have shown nodes as –
  - Monitor
  - Modem
  - Caching server
  - Server
- The application is assumed to be a web-based application, which is deployed in a clustered environment using server 1, server 2, and server 3. The user connects to the application using the Internet. The control flows from the caching server to the clustered environment.

Deployment diagram example…



Deployment diagram of an order management system



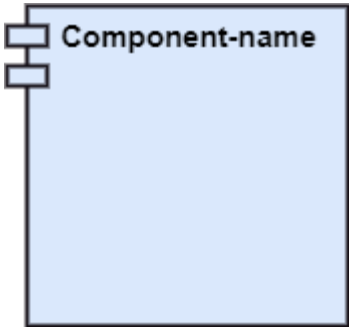
When to use a Deployment Diagram?

- The deployment diagram is mostly employed by network engineers, system administrators, etc.
- The software applications are quite complex these days, as they are standalone, distributed, web-based, etc. So, it is very necessary to design efficient software.
  - Deployment diagrams can be used for the followings:
  - To model the network and hardware topology of a system.
  - To model the distributed networks and systems.
  - Implement forwarding and reverse engineering processes.
  - To model the hardware details for a client/server system.
  - For modeling the embedded system.

UML Component Diagram

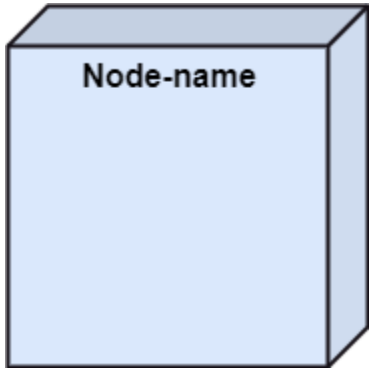
- A component diagram is used to break down a large object-oriented system into the smaller components, so as to make them more manageable. It models the physical view of a system such as executables, files, libraries, etc.
- It visualizes the relationships as well as the organization between the components present in the system. It helps in forming an executable system.
- A component is a single unit of the system, which is replaceable and executable. The implementation details of a component are hidden, and it necessitates an interface to execute a function. It is like a black box whose behavior is explained by the provided and required interfaces.

Notation of a Component Diagram

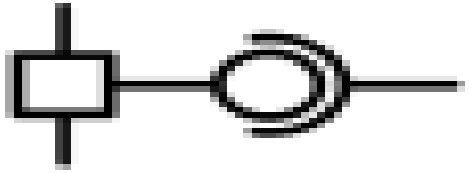


a) **Component** :A component is a logical unit block of the system, a slightly higher abstraction than classes.

b) A node: Nodes **represent hardware devices such as computers, sensors, and printers, as well as other devices that support the runtime environment of a system.** Communication paths and deploy relationships model the connections in the system.



c) **Interface:** An interface (small circle or semi-circle on a stick) describes a group of operations used (required) or created (provided) by components. A full circle represents an interface created or provided by the component. A semi-circle represents a required interface, like a person's input.



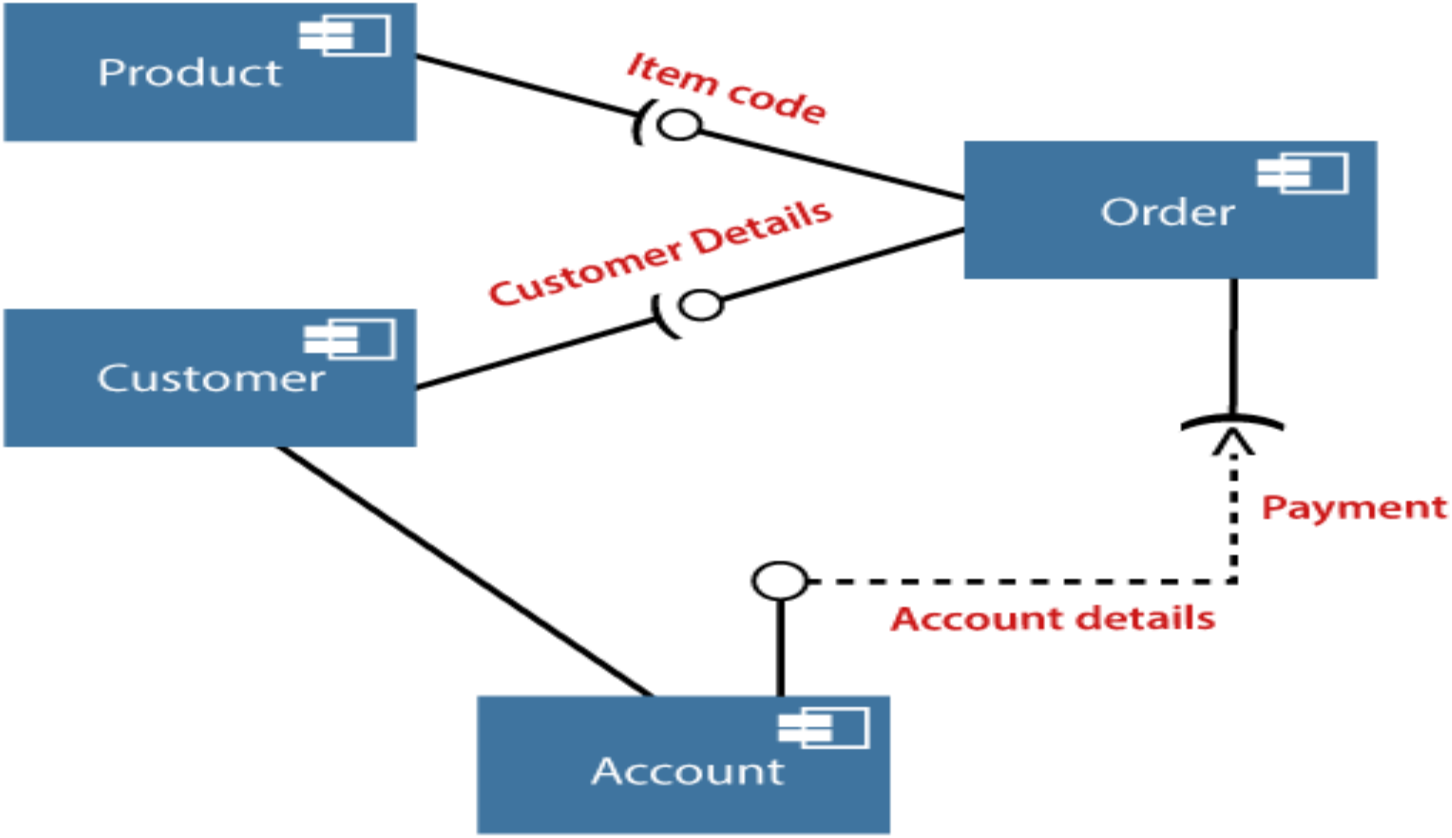
Purpose of a Component Diagram

- It describes all the individual related components that are used to make the functionalities. It visualizes the physical components inside the system. The components can be a library, packages, files, etc.
- The component diagram also describes the static view of a system. The collection of component diagrams represents a whole system.
- The main purpose of the component diagram are enlisted below:
  - It envisions each component of a system.
  - It constructs the executable by incorporating forward and reverse engineering.
  - It depicts the relationships and organization of components.

Why use Component Diagram?

- The Following are some reasons for the requirement of the component diagram:
  - It portrays the components of a system at the runtime.
  - It is helpful in testing a system.
  - It envisions the links between several connections.
  - To divide a single system into multiple components according to the functionality. Or It used to identifies related subsystems of the system
  - To represent the component organization of the system.

Example of a Component Diagram: for online shopping system



Where to use Component Diagrams?

- It represents the physical components of a system, or we can say it portrays the organization of the components inside a system. The components, such as libraries, files, executables, etc. are first needed to be organized before the implementation.
- The component diagram can be used for the followings:
  - To model the components of the system.
  - To model the schemas of a database.
  - To model the applications of an application.
  - To model the system's source code.

Library Management System

Problem Statement:

- The case study titled Library Management System is library management software for the purpose of monitoring and controlling the transactions in a library. This case study on the library management system gives us the complete information about the library and the daily transactions done in a Library. We need to maintain the record of new s and retrieve the details of books available in the library which mainly focuses on basic operations in a library like adding new member, new books, and up new information, searching books and members and facility to borrow and return books. It features a familiar and well thought-out, an attractive user interface, combined with strong searching, insertion and reporting capabilities. The report generation facility of library system helps to get a good idea of which borrowed by the members, makes users possible to generate hard copy.

- THANK YOU！